2.软件

软件是计算机系统中与硬件相互依存的部分，它是包括程序、数据及相关文档的完整集合

7.软件工程

软件工程是研究和应用如何以系统化的、规范的、可度量的方法去开发、运行和维护软件，即把工程化方法应用到软件上。

17.软件生存周期

软件生存周期是指软件产品从考虑其概念开始到该软件产品交付使用，直至最终退役为止的整个过程，一般包括**计划、分析、设计、实现、测试、集成、交付、维护**等阶段。

22.软件过程

软件过程是人们用于开发和维护软件及其相关过程的一系列活动，包括软件工程活动和软件管理活动。

27.软件基本过程

软件过程（Software Process）是指一套关于项目的阶段、状态、方法、技术和开发、维护软件的人员以及相关 Artifacts（计划、文档、模型、编码、测试、手册等）组成。软件过程可概括为三类：**基本过程类、支持过程类和组织过程类**。基本过程类包括获取过程、供应过程、开发过程、运作过程、维护过程和管理过程。

32.软件支持过程

支持过程类包括文档过程、配置管理过程、质量保证过程、验证过程、确认过程、联合评审过程、审计过程以及问题解决过程。

37.软件组织过程

组织过程类包括**基础设施过程、改进过程以及培训过程**。

97.过程规范（未找到答案）

对软件从立项、设计、开发、实施、维护各阶段中的每一个过程根据一定的标准进行规范。

38.软件度量

是对软件开发项目、过程及其产品**进行数据定义**、收集以及分析的持续性定量化过程，目的在于对此加以理解、预测、评估、控制和改善。

68.软件质量

与软件产品满足规定的和隐含的需求的能力有关的特性或特性的全体。或“所有描述计算机软件优秀程度的特性的组合。”

12.软件危机

软件危机是指在计算机软件的开发和维护过程中所遇到的一系列严重问题

83.软件可靠性

（1）在规定的条件下，在规定的时间内，软件不引起系统失效的概率。
（2）在规定的时间周期内，在所述条件下程序执行所要求的功能的能力。

47.软件能力成熟度模型

它是对于软件组织在定义、实施、度量、控制和改善其软件过程的实践中各个发展阶段的描述。CMM 的核心是把软件开发**视为一个过程**，并根据这一原则对软件开发和维护进行过程监控和研究，以使其更加科学化、标准化、使企业能够更好地实现商业目标。

93.软件质量模型（未找到答案）

随着计算机技术及其应用的日益普及,各行各业对软件的依赖性越来越大;同时,由于软件质量问题导致的不良后果也越来越多。因此,软件质量的重要性日益突出。如何保证软件质量已经成为软件开发过程中必须贯穿始终加以考虑的重要问题。然而,质量是一个复杂且难以琢磨的概念。对于不同的人或不同的应用系统,质量的含义和要求不尽相同。并且,软件质量属性是多方面的。一个软件的质量往往涉及到许多不同的质量属性,不同类型的软件所关注的质量属性也不尽相同。因此,为了更好地理解、预测和评价软件和信息系统的质量,人们建立了各种质量模型来衡量软件的质量。

基于软件的特性和大家所关心的质量问题，形成一套大家都能接受衡量软件质量高低的一种模型。

96.软件开发过程模型

软件过程模型 是指软件开发全部过程、活动和任务的结构框架。
软件过程是为了获得高质量的软件产品所需要完成的一系列任务的框架，它规定了完成各项任务的工作步骤。通常使用生命周期模型简洁地描述软件过程。生命周期模型规定了把生命周期划分成哪些阶段及各个阶段的执行顺序，因此，也称为过程模型。

常用的过程模型有：
瀑布模型（需求完全确定）
渐进式开发（逐步迭代）模型：原型模型、增量模型、螺旋模型
瀑布模型的特点是：
每个阶段都必须完成规定的文档，没有交出合格的文档就是没有完成该阶段的任务。
每个阶段结束前都要对所完成的文档进行评审，以便迟早发现问题，改正错误。

1）规定了阶段之间自上而下、相互衔接的固定次序，上一阶段的成果作为下一阶段的输入
2）每一个阶段完成之后，必须对阶段性成果进行评审，确认之后才可进入下一阶段。
瀑布模型是由文档驱动的。这个事实也是它的一个缺点，在可运行的软件产品交付给用户之前，用户只能通过文档来了解产品是什么样的。最终开发出的产品可能并不是用户真正需要的。
快速原型模型：快速原型模型的第一步是建造一个快速原型，实现客户或未来的用户与系统的交互，用户或客户对原型进行评价，进一步细化待开发软件的需求。通过逐步调整原型使其满足客户的要求，开发人员可以确定客户的真正需求是什么；第二步则在第一步的基础上开发客户满意的软件产品。
螺旋模型：它将瀑布模型和快速原型模型结合起来，强调了其他模型所忽视的风险分析，特别适合于大型复杂的系统。螺旋模型沿着螺线旋转，分 4 个方面的活动：**制定计划、风险分析、实施工程、客户评估。**沿螺旋线自内向外每旋转一圈，便开发出一个更为完善的、新的软件版本。螺旋模型由风险驱动，强调可选方案和约束条件从而支持软件的重用，有助于将软件质量作为特殊目标融入产品开发之中。
增量模型：把软件产品作为一系列的增量构件来设计、编码、集成和测试。每个构件由多个相互作用的模块构成，并且能够完成特定的功能。使用增量模型时，第一个增量构件往往实现软件的基本需求，提供最核心的功能。强调每一个**增量均发布一个可操作产品**。

喷泉模型：喷泉模型对软件复用和生存周期中多项开发活动的集成提供了支持，以面向对象的软件开发方法为基础，它适合面向对象的开发方法。它克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。喷泉模型使开发过程具有迭代性和无间隙性。系统某个部分常常重复工作多次，相关功能在每次迭代中随之加入演化的系统。无间隙是指在分析、设计和实现等开发活动之间不存在明显的边界。

62/99.过程模型

所谓软件过程模型就是一种开发策略，这种策略针对软件工程的各个阶段提供了一套范形，使工程的进展达到预期的目的。对一个软件的开发无论其大小，我们都需要选择一个合适的软件过程模型，这种选择基于项目和应用的性质、采用的方法、需要的控制，以及要交付的产品的特点。一个错误模型的选择，将迷失我们的开发方向

49.非功能性需求

非功能需求（non-functional requirement）是从各个角度对系统的约束和限制，反映了应用对软件系统质量和特性的额外要求。

142、低代码

低代码通过可视化编程，具有不同经验、水平的开发人员可以通过图 形化用户界面，使用拖拽组件和模型驱动的逻辑来创建网页和移动应用程序。低代码能够在合适的业务场景下实现大幅度的**提效降本**。
（简单：代码码=可视化+少量或 0 代码+快速生成）。
低代码开发平台（LCDP）是无需编码（0 代码）或通过少量代码就可以快速生成应用程序的开发平台。

143、过程改进

为了满足新的目标，过程拥有者对过程所执行的一系列改进活动。包括对已有某个组织内软件过程的标识、分析和改变。
过程改进是一种组织管理的方法，涉及对软件开发过程的优化和改进，旨在通过一系列活动、方法、实践及转换来提
高工作流程效率、降低成本、提高用户满意度。

144、保护性活动

保护性活动是软件开发团队为了维护完整开发任务的进度、质量、更改和风险而遵循的一系列步骤或过程。

84/89.数据流图/上下文数据流图

数据流图：简称 DFD，它从数据传递和加工角度，以图形方式来表达系统的逻辑功能、数据在系统内部的**逻辑流向和逻辑变换过程**，是**结构化系统分析方法**的主要表达工具及用于表示软件模型的一种图示方法。

94.业务流程图

业务流程图是一种描述系统内各单位、人员之间**业务关系、作业顺序和管理信息流向的图表**，利用它可以帮助分析人员找出业务流程中的不合理流向，它是物理模型。业务流程图主要是描述业务走向。业务流程图描述的是完整的业务流程，**以业务处理过程为中心，一般没有数据的概念**

69.体系结构

体系结构包括一组部件以及部件之间的联系。

42.过程框架

定义了**若干小的框架活动**，为完整的软件开发过程建立基础

43.功能点分析

是面向功能的软件度量方法，**注意力集中于软件系统的“功能性”和“实用性”**

44.结构化程序设计

进行大型程序设计的一种方法。整个程序是由各个部分按一定的层次结构组织起来的。每一层次中规定只使用几种基本的程序结构。从最高层开始，依次向下，逐步求精。设计出的程序条理清楚、容易理解、容易调试与维护

48.工作分解结构

以可交付成果为导向对项目要素进行的分组，它归纳和定义了项目的整个工作范围每下降一层代表对项目工作的更详细定义。

74.决策表

决策表又称判断表，是一种呈**表格状的图形工具，适用于描述处理判断条件较多，各条件又相互组合、有多种决策方案的情况**。精确而简洁描述复杂逻辑的方式，将多个条件与这些条件满足后要执行动作相对应。但不同于传统程序语言中的控制语句，决策表能将多个独立的条件和多个动作直接的联系清晰的表示出来

78.文本复杂性度量（未找到答案）

软件的复杂性。分为四类：结构、数据结构、算法、文档复杂性度量。

79.数据字典

精确地、严格地定义了**每个与系统相关的数据元素**，并以字典式顺序将它们组织起来，使得用户和分析员对所有的输入、输出、存储成分和中间计算有共同的理解。

101. 类整体-部分结构图

如建立一个时钟类，这个类中的属性如秒、分钟、小时等，加上对属性的操作如秒记时，分钟记时，小时记时等，就是一个类的整体部分结构。

15.面向对象方法特征有哪些？

封装性、继承性、多态性

1、封装性

封装是一种信息隐蔽技术，它体现于类的说明，是对象的重要特性。封装使数据和加工该数据的方法（函数）封装为一个整体，以实现独立性很强的模块，使得用户只能见到对象的外特性（对象能接受哪些消息，具有那些处理能力），而对象的内特性（保存内部状态的私有数据和实现加工能力的算法）对用户是隐蔽的。封装的目的在于把对象的设计者和对象者的使用分开，使用者不必知晓行为实现的细节，只须用设计者提供的消息来访问该对象。

2、继承性

继承性是子类自动共享父类之间数据和方法的机制。它由类的派生功能体现。一个类直接继职其它类的全部描述，同时可修改和扩充。

继承性具有传达室递性。继承分为单继承（一个子类只有一父类）和多重继承（一个类有多个父类）。类的对象是各自封闭的，如果没继承性机制，则类对象中数据、方法就会出现大量重复。继承不仅支持系统的可重用性，而且还促进系统的可扩充性。

3、多态性

对象根据所接收的消息而做出动作。同一消息为不同的对象接受时可产生完全不同的行动，这种现象称为多态性。利用多态性用户可发送一个通用的信息，而将所有的实现细节都留给接受消息的对象自行决定，如是，同一消息即可调用不同的方法。例如：Print 消息被发送给一图或表时调用的打印方法与将同样的 Print 消息发送给一正文文件而调用的打印方法会完全不同。多态性的实现受到继承性的支持，利用类继承的层次关系，把具有通用功能的协议存放在类层次中尽可能高的地方，而将实现这一功能的不同方法置于较低层次，这样，在这些低层次上生成的对象就能给通用消息以不同的响应。在 OOPL 中可通过在派生类中重定义基类函数（定义为重载函数或虚函数）来实现多态性。

综上可知，在 OO 方法中，**对象和传递消息分别表现事物及事物间相互联系的概念**。类和继承是是适应人们一般思维方式的描述范式。方法是允许作用于该类对象上的各种操作。这种对象、类、消息和方法的程序设计范式的基本点在于对象的封装性和类的继承性。通过封装能将对象的定义和对象的实现分开，通过继承能体现类与类之间的关系，以及由此带来的动态联编和实体的多态性，从而构成了面向对象的基本特征。

55.面向对象

首先根据客户需求抽象出业务对象；**然后对需求进行合理分层，构建相对独立的业务模块；**之后设计业务逻辑，**利用多态、继承、封装、抽象的编程思想，**实现业务需求；最后通过整合各模块，**达到高内聚、低耦合的效果**，从而满足客户要求。

85.对象/类

类是对象的抽象，而对象是类的具体实例。类是抽象的，不占用内存，而对象是具体的，占用存储空间。

54.信息隐蔽

信息隐蔽指在设计和确定模块时,使得一个模块内包含信息(过程或数据),对于不需要这些信息的其他模块来说,是不能访问的。在面向对象方法中，信息隐蔽是通过对象的**封装性来实现的。信息隐蔽的概念与模块的独立性直接相关。**

80.封装

隐藏**对象的属性和实现细节**，仅对外公开接口,控制在程序中属性的读和修改的访问级别。

封装就是将抽象得到的数据和行为（或功能）相结合，形成一个有机的整体，也就是将数据与操作数据的源代码进行有机的结合，形成“类”，**其中数据和函数都是类的成员**。

75.继承

继承是指一个**对象**直接使用另一**对象**的**属性和方法**。

65.多态性

同一操作作用于不同**的类的实例**，将产生不同的执行结果，即不同类的**对象**收到相同的消息时，得到不同的结果。对象根据所接受的消息而做出动作，同样的消息被不同的对象接收时可能导致完全不同的行为，这种现象称为多态性。

50.统一建模语言 UML

它是一个支持模型化和软件系统开发的图形化语言,为软件开发的所有阶段提供模型化和可视化支持，包括由需求分析到规格，到构造和配置。是一种面向对象的建模语言，**它是运用统一的、标准化的标记和定义实现对软件系统进行面向对象的描述和建模。**

70.消息

消息，软件对象之间进行交互作用和通讯是利用消息的。在面向对象的程序设计中，消**息是指一个类实例和另一个类实例之间传递的信息**。

5. 分布图

分布图反映了系统中**软件和硬件的物理架构**,表示系统运行时的处理节点以及节点中组件的配置。

10.组件图

组件图描述组件以及它们之间的关系，表示**系统的静态实现视图**。

15.活动图

活动图反映系统中从一个活动到另一个活动的流程，**强调对象间的控制流程**。

45.用例图

用例图定义了系统的功能需求，它完全是从系统的外部观看系统功能，并不描述系统内部对功能的具体实现。

40.类图

类图描述系统的**静态结构**，表示系统中的**类以及类与类之间的关系**。

30.状态图

状态图表示一个状态机，**强调对象行为的事件顺序**。

35.对象图

对象图描述了一组对象以及它们之间的关系，表示**类的对象实例**。

20.协作图

协作图表示一组**对象之间的动态协作关系**，反映收发消息的对象的结构组织。

25.时序图

时序图表示一组对象之间的动态协作关系，反映对象之间发送消息的时间顺序。

90.静态视图

是因为它不描述与时间有关的系统行为，此种行为在其他视图中进行描述。静态视图主要是**由类及类间相互关系构成**，这些相互关系包括：**关联、泛化和各种依赖关系**，如使用和实现关系

95.动态视图（未找到答案）

就是用图形化的方法表示各种状态随时间变化的状态。动态视图包括：顺序图、交互图、状态图、活动图。

60.主动对象

主动对象内部包含一个**线程**，可以自动完成动作或改变状态。主动对象是内部拥有自己的控制线程的对象。

105、构件图（组件图）

构件图是以代码构件为单位表示代码的物理结构，及构件之间的依赖关系。

106、部署图

部署图是表示系统中硬件和软件的物理结构，及各部分之间的连接方式。

94.软件测试

按是否查看程序内部结构分为：

黑盒测试：只关心输入和输出的结果

着眼于软件的外部特性，而不开了软件的内部逻辑结构和处理过程，只依据程序的需求规格说明书，检查程序的功能是否符合他的功能说明，故又称为功能测试。

主要检查：程序的功能是否正确或完善；数据的输入能否正确接收，输出是否正确；能否保证外部信息的完整性。
方法有：等价类划分法，边界值法，错误推测法，因果图法，判定表法，状态迁移法，正交试验法
白盒测试：去研究里面的源代码和程序结构
白盒测试也称结构测试或逻辑驱动测试，它是知道产品内部工作过程，可通过测试来检测产品内部动作是否按照规格说明书的规定正常进行，按照程序内部的结构测试程序，检验程序中的每条通路是否都有能按预定要求正确工作，而不顾它的功能。
语句覆盖：设计足够的测试用例，使**每个语句**至少执行一次
判定覆盖：设计足够的测试用例，使**每判定**的取真分支和取假分支至少执行一次
条件覆盖：设计足够的测试用例，使每个判断中**每个条件的可能取值至少满足一次**
条件组合覆盖：设计足够的测试用例，使所有判定中**各条件判断结果的所有组**合至少出现一次。
路径覆盖：设计足够多的测试用例，覆盖程序中所有可能的路径。覆盖程度最高
按是否运行程序分为：
静态测试：是指不实际运行被测软件，而只是静态地检查扫描程序代码、界面或文档可能存在的错误的过程。
动态测试：是指实际运行被测程序，输入相应的测试数据，检查输出结果和预期结果是否相符的过程
按阶段划分：
单元测试：对软件中的最小可测试单元进行检查和验证。
集成测试：是单元测试的下一阶段，是指将通过测试的单元模块组装成系统或子系统，再进行测试，重点测试不同模块的接口部门。集成测试就是用来检查各个单元模块结合到一起能否协同配合，正常运行。
系统测试：指的是将整个软件系统看做一个整体进行测试，包括对功能、性能，以及软件所运行的软硬件环境进行测试。系统测试的主要依据是《系统需求规格说明书》文档。
验收测试：指的是在系统测试的后期，以用户测试为主，或有测试人员等质量保障人员共同参与的测试，它也是软件正式交给用户使用的最后一道工序。

其他还有回归测试、冒烟测试、随机测试
黑盒测试包括功能测试和性能测试；
功能测试有：逻辑功能测试、界面测试、易用性测试、安装测试、兼容测试；
性能测试有：一般性能测试、稳定性测试、压力测试、负载测试

76.测试配置（未找到答案）

配置测试用于测试和验证软件，在不同的软件和硬件配置中进行运行。配置测试就是测试软件是否和系统的其他与之交互的元素之间兼容，如浏览器、操作系统、硬件等，验证被测软件在不同的软件和硬件配置中的运行情况。

71.压力测试

在软件工程中，压力测试是对系统不断施加压力的测试，是通过确定一个系统的瓶颈或者不能接收的性能点，来获得系统能提供的最大服务级别的测试。

61.确认测试

检查已实现的软件是否满足了需求规格说明中所确定的各种需求，以及软件配置是否完全、正确。（过程：功能性测试---软件配置复查---验收测试----α测试和β测试）

91.Beta 测试

Beta 测试由软件的最终用户们在一个或多个客房场所进行。与 Alpha 测试不同，开发者通常不在 Beta 测试的现场，因 Beta 测试是软件在开发者不能控制的环境中的“真实”应用。用户 Beta 测试过程中遇到的一切问题（真实在或想像的），并且定期把这些问题报告给开发者。接收到在 Beta 测试期间报告的问题之后，开发者对软件产品进行必要的修改，并准备向全体客户发布最终的软件产品

36.基本路径测试

是在程序控制流图的基础上，通过分析**控制构造的环路复杂性**，导出基本可执行路径集合，从而设计测试用例的方法。设计出的测试用例要保证在测试中程序的每个可执行语句至少执行一次

41.黑盒测试

黑盒测试也称功能测试或数据驱动测试，它是在已知产品所应具有功能的情况下，通过测试来检测每个功能是否都能正常使用。

黑盒测试也称功能测试，它是通过测试来检测每个功能是否都能正常使用。在测试中，把程序看作一个不能打开的黑盒子，在完全不考虑程序内部结构和内部特性的情况下，在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当地接收输入数据而产生正确的输出信息。黑盒测试着眼于程序外部结构，不考虑内部逻辑结构，**主要针对软件界面和软件功能进行测试。**

26.等价类划分

等价类划分法是一种典型的、重要的**黑盒测试方法**，它将程序所有可能的输入数据（有效的和无效的）划分成若干个等价类。然后从每个部分中选取具有代表性的数据当做测试用例进行合理的分类，测试用例由有效等价类和无效等价类的代表组成，从而保证测试用例具有完整性和代表性。

31.边界值测试

针对**各种边界情况**设计测试用例

96.因果图

因果图是根据分析时对各种条件可能产生的结果的一种对照分析法。 **用于软件黑盒测试.**
鱼骨图是由日本管理大师石川馨先生所发明出来的，故又名石川图。鱼骨图是一种发现问题“根本原因”的方法。其特点是简捷实用，深入直观。它看上去有些象鱼骨，问题或缺陷（即后果）标在"鱼头"外。在鱼骨上长出鱼刺，上面按出现机会多寡列出产生生产问题的可能原

因。鱼骨图有助于说明各个原因之间如何相互影响。它也能表现出各个可能的原因是如何随时间而依次出现的。这有助于着手解决问题。

46.白盒测试

98.基于时间的缺陷到达模式（未找到答案）

是一种软件过程质量测试方法，是基于时间的缺陷到达模式，缺陷到达模式可以提供更多的信息，有时即使得到的整体缺陷率是一样的，但其质量差异可能较大，原因就是缺陷到达的时间不同，越多的缺陷到达越早，则测试过程质量就越好。

59.内聚

内聚是模块功能强度（**一个模块内部各个元素彼此结合的紧密程度**）的度量。

64.耦合

耦合是模块之间的相对独立性（互相连接的紧密程度）的度量。

4.通信聚合

通信内聚：访问相同数据的所有操作被定义在一个类中。一般来说，这些类只着眼于数据的查询。访问和存储

9.信息聚合（未找到答案）

就是把目前所掌握的信息资料合到一块，围绕某主题，把极度分散、高度相关、前后依存的信息碎片，迅速、及时的整合成完整的、有参考价值的信息。

14.数据聚合（未找到答案）

结合来自不同节点的数据，通过去除冗余，减少信息的传输量，从而达到节省能量的目的。

19.控制耦合

当操作 A 调用操作 B，并且向 B 传递了一个**控制标记**时，就会发生此种耦合

24.标记耦合

当**类 B 被声明**为类 A 某一操作中的**一个参数类型**时会**发生此种耦合**。由于**类 B 现在作为类 A 定义的一部分，**所以修改系统就会变得更为复杂

29.数据耦合

当操作需要传递较长的数据参数时就会发生此种耦合。随着类和构件之间通信“带宽”

的增长以及接口复杂性的增加，测试和维护就会越来越困难

3.项目

项目是一件事情、一项独一无二的任务，也可以理解为是在一定的时间和一定的预算内所要达到的预期目的。项目是指在一定的约束条件下（主要是限定时间、限定资源），具有明确目标的一次性任务。项目是一系列具有特定目标,有明确开始和终止日期，资金有限，消耗资源的活动和任务。

33.项目整体管理

也叫集成管理，保证项目各个要素相互协调所需要的管理过程，它需要在相互影响的项目目标和方案中作出平衡，以满足或超出项目干系人的需求和愿望

28.项目范围管理

项目范围管理是指对项目包括什么与不包括什么进行定义并控制的过程。这个过程用于确保项目组和项目干系人对做为项目结果的项目产品以及生产这些产品所用到的过程有一个共同的理解。

8.配置管理

配置管理(Configuration Management，CM)是通过技术或行政手段对软件产品及其开发过程和生命周期进行控制、规范的一系列措施。配置管理的目标是记录软件产品的演化过程，确保软件开发者在软件生命周期中各个阶段都能得到精确的产品配置。

58.项目计划评审技术

计划评审技术就是工程项目当作一种系统，用网络图或者表格或者矩阵来表示各项具体工作的先后顺序和相互关系，以时间为中心，找出从开工到完工所需要时间的最长路线，并围绕关键路线对对系统进行统筹规划，合理安排以及对各项工作的完成进度进行严密的控制，以达到用最少的时间和资源消耗来完成系统预定目标的一种计划与控制方法。

23.里程碑

里程碑一般是项目中完成阶段性工作的标志，标志着上一个阶段结束、下一个阶段开始，将一个过程性的任务用一个结论性的标志来描述，明确任务的起止点。一系列的起止点就构成了引导整个项目进展的里程碑。里程碑定义了当前阶段完成的标准和下一新阶段启动的条件和前提。

13.配置项

软件配置项是在软件工程中过程中创建的信息，凡是纳入配置管理范畴的工作成果都是

配置项；一个纯软件的 CIS 通常也称为软件配置。

18.基线

是一个软件**配置管理**的概念，它能够帮助我们在**不严重阻碍合理变更的条件下控制变更，**IEEE 中这样定义基线：**已经通过正式审评和批准的规格说明或产品**，他可以作为进一步开发的基础，并且只有通过正式的变更控制规程才能修改它

34.预防性维护

预防性维护是指采用先进的软件工程方法对需要维护的软件或软件中的某一部分重新进行设计、编制和测试，提高软件的可维护性和可靠性等，为以后进一步改进软件打下良好基础。

为了降低设备失效或功能退化的概率，按预定的时间间隔或规定的标准进行的维护。

39.适应性维护

随着计算机技术的飞速发展和更新换代，软件系统所需的外部环境或数据环境可能会更新和升级，如操作系统或数据库系统的更换等。为了使软件系统适应这种变化，需要对软件进行相应的修改，这种维护活动称为适应性维护。

92.极限编程

极限编程是一个轻量级的、灵巧的软件开发方法；同时它也是一个非常严谨和周密的方法。它的基础和价值观是**交流、朴素、反馈和勇气**；即，任何一个软件项目都可以从四个方面入手进行改善：**加强交流；从简单做起；寻求反馈；勇于实事求是**。XP 是一种近螺旋式的开发方法，它将复杂的开发过程分解为一个个相对比较简单的小周期；通过积极的交流、反馈以及其它一系列的方法，开发人员和客户可以非常清楚开发进度、变化、待解决的问题和潜在的困难等，并根据实际情况及时地调整开发过程

分布图反映了系统中**软件和硬件的物理架构**，表示系统运行时的处理节点以及节点中组件的配置。

53.COCOMO 模型

构造性成本模型，是一种利用经验模型进行成本估算的方法。

它是一种精确、易于使用的，**基于模型的成本估算方法**。最早由勃姆（Boehm）于 1981 年提出。从本质上说是一种参数化的项目估算方法，参数建模是把项目的某些特征作为参数，通过建立一个数字模型预测项目成本（类似于居住面积作为参数计算的整体的住房成本）。

52.个体软件过程

就是为使软件工程师更好地工作而设计的一个框架，它指出如何估计和计划工作，如何按照这些计划来跟踪自己的性能，以及如何提高程序的质量。

57.团队软件过程

团队软件过程是为开发软件产品的开发团队提供指导，TSP 的早期实践侧重于帮助开发团队改善其质量和生产率，以使其更好的满足成本及进度的目标。加上 PSP 帮助高绩效的工程师在一个团队中工作，来开发有质量保证的软件产品，生产安全的软件产品，改进组织中的过程管理。

63.RMMM 计划（Risk Mitigation, Monitoring and Management Plan）

翻译为：风险缓解，监测和管理计划，又称软件项目风险管理，是软件项目管理的重要内容。在进行软件项目风险管理时，要辩识风险，评估它们出现的概率及产生的影响，然后建立一个规划来管理风险。风险管理的主要目标是预防风险。

软件项目风险是指在软件开发过程中遇到的预算和进度等方面的问题以及这些问题对软件项目的影响。软件项目风险会影响项目计划的实现，如果项目风险变成现实，就有可能影响项目的进度，增加项目的成本，甚至使软件项目不能实现。

67.统一过程

是软件工程的过程。**它提供了在开发组织中分派任务和责任的纪律化方法。**它的目标是在可预见的日程和预算前提下，确保满足最终用户需求的高质量产品。

统一过程模型是一种**“用例驱动，以体系结构为核心，迭代及增量”**的软件过程框架，由 UML 方法和工具支持。

73.结构复杂性度量（未找到答案）

软件度量的根本目的是为了管理的需要,利用度量来改进软件过程。人们是无法管理不能度量的事物。没有对软件过程的可见度就无法管理;而没有对见到的事物有适当的度量或适当的准则去判断、评估和决策,也无法进行优秀的管理。

结构复杂性度量就是为了管理需要，通过工程化的方法去衡量软件内部的结构的复杂性。

88.错误播种（植入）模型（未找到答案）

是一种软件错误模拟技术，即在软件系统中人为的播种一定数量的已知类型的错误或错误数目，之后通过让他人检测，看看检测出来的错误数中包含多少植入的错误，从而间接地计算出软件中包含的错误数或者测试人员的水平。

当然也可以通过播种特定位置特定类型的错误来评估软件的性能，研究特定类型的软件错误的特性。

100.领域分析

领域分析的目的是**支持软件重用**，它产生的是描述了特定领域中各系统共性的领域模型。

领域分析的过程由领域分析的前期准备、分析领域知识和形成重用成分三部分组成。

面向对象分析方法是领域分析的一个很好工具。

2.简述软件开发的可行性研究的目的和内容。

可行性研究是指对一个计划、项目或想法进行详尽的调查和评估，以确定其是否可行或可实施。

可行性研究的目的在于确定项目是否有实施的必要性和是否具备实施的条件（如技术、资源、法律等）。

内容涵盖技术可行性（能否实现功能）、**经济可行性**（成本与收益是否合理）、**社会可行性**（是否符合道德和社会影响）、**法律可行性**（是否合规）。

作用：对于确保项目的顺利实施和投资回报有着至关重要的作用

33.需求分析的任务是什么？＊

需求分析是将用户需求陈述转化为完整的需求定义，继而形成规范化的需求规格说明的过程。其基本任务是准确定义新系统的功能，主要任务有：

⑴需求分析是借助于当前系统导出目标系统的逻辑模型，解决目标系统做什么；

⑵深入描述软件的功能和性能，确定软件设计的限制和软件同其它系统元素的接口细节，定义软件的其它有效性需求；

⑶软件需求规格说明还为评价软件质量提供依据；

37.如何安排软件开发进度，应该考虑哪些问题？

软件工程中，软件开发进度的安排是一个重要的管理活动，应该考虑以下几个问题：

- 软件开发的目标和范围：明确软件要实现的功能、性能、质量等需求，以及软件的交付时间和成本等约束。
- 软件开发的流程和阶段：选择合适的软件开发模型，如瀑布模型、迭代模型、敏捷模型等，划分软件开发的主要阶段，如需求分析、设计、编程、测试、部署等，以及每个阶段的输入、输出、标准和评审等。
- 软件开发的资源和人员：评估软件开发所需的硬件、软件、人力等资源，分配和协调项目团队的角色和职责，如项目经理、分析师、设计师、程序员、测试员等，以及他们之间的沟通和协作方式。
- 软件开发的风险和变更：识别和分析软件开发过程中可能遇到的风险，如需求变更、技术难题、人员流失等，制定风险应对策略和变更控制机制，以及风险和变更的跟踪和报告方式。

41.原型开发由哪些步骤？

原型开发是一种快速开发软件的方法，主要由以下几个步骤组成：

- 确认用户基本需求，需求分析师与用户合作完成，可以不完整，易于描述，大型系统应该形成需求文档。
- 设计系统初始原型，实现一个可运行的系统，原则如下：
  - 集成原则：利用现有的系统和原型构成，需要原型工具。
  - 最小原则：只实现最基本的功能，不考虑细节和优化。
  - 可变原则：保持系统的灵活性和可修改性，方便后续改进。

- 评估和修改原型，将原型展示给用户，收集用户的反馈和建议，根据用户的需求进行修改和完善，直到用户满意为止。
- 转换为目标系统，将原型转换为正式的软件系统，需要考虑系统的稳定性、可靠性、安全性、效率等方面。

43.原型开发技术的目的，特点是什么？

解决用户与开发者之间的鸿沟,以原型(软件产品的样品)为共同语言，实现用户与开发者双向沟通。

容易地确定系统的性能，确认各项主要系统服务的可应用性，确认系统设计的可行性，确认系统作为产品的结果。

软件开发人员和用户之间的理解偏差在功能展现时显露出来在原型设计中发现需求的不完善和不一致可迅速地展现一个简单的应用系统通过原型系统可以导出系统需求原型系统可以支持用户培训和系统测试。

51.原型具有什么特点？

1）遵循了人们认识事物的客观规律，易于掌握和接受
2）将模拟的手段引入系统分析的初始阶段，沟通了人们（用户和开发人员）的思想，缩短了用户和系统分析人员之间的距离，解决了结构化方法中最难于解决的一环。强调用户参与、描述、运行、沟通。
3）充分利用最新的软件工具，摆脱了传统的方法，使系统开发的时间、费用大大地减少，效率、技术等方面都大大地提高。强调软件工具支持。

46.原型开发包括那几类，目的是什么？

1）抛弃式原型开发是弄清需求和为风险评估提供额外的信息。
2）演化式原型开发先给出一个系统的最初实现，让用户去使用和评价，不断进行细化和完善，经过多次反复形成最后完善的应用系统。
3）递增式原型开发减少修改的范围，实现易于控制和管理。

56. 软件开发中，为什么要进行阶段评审？

进行阶段评审的主要原因包括以下几点：

确保项目达到既定目标和标准:阶段评审的主要目的是确保项目在每个阶段都能够达到既定的目标和标准。通过评审，可以及时发现和解决问题，保证项目的顺利进行。

提高项目质量和效率：阶段评审可以有效地发现项目中存在的问题，提出改进建议，为项目的后续工作提供有力支持。同时,评审工作也能够提高团队成员的工作积极性和责任感，促进团队的协作和沟通，提升项目的整体质量和效率。

风险管理：阶段评审关注产品或项目的整体开发进度、当前状态及潜在风险管理。通过评审，可以及时发现潜在的风险，并采取相应的措施进行管理，确保项目按计划顺利进行

66.什么是需求工程？

需求工程是指应用已证实有效的技术、方法进行需求分析，确定客户需求，帮助分析人员理解问题并定义目标系统的所有外部特征的一门学科。

它通过合适的工具和记号系统地描述待开发系统及其行为特征和相关约束，形成需求文档，并对用户不断变化的需求演进给予支持。RE 可分为系统需求工程（如果是针对由软硬件共同组成的整个系统）和软件需求工程（如果仅是专门针对纯软件部分）。软件需求工程是一门分析并记录软件需求的学科，它把系统需求分解成一些主要的子系统和任务，把这些子系统或任务分配给软件，并通过一系列重复的分析、设计、比较研究、原型开发过程把这些系统需求转换成软件的需求描述和一些性能参数。

需求工程是一个不断反复的需求定义、文档记录、需求演进的过程，并最终在验证的基础上冻结需求。80 年代，HerbKrasner 定义了需求工程的五阶段生命周期：需求定义和分析、需求决策、形成需求规格、需求实现与验证、需求演进管理。近来，MatthiasJarke 和 KlausPohl 提出了三阶段周期的说法：获取、表示和验证。

61.简述软件生命周期模型在软件开发中的作用。

软件生命周期是软件工程的一个重要的概念。把整个软件生命周期划分为若干个较小的阶段,每个阶段都有相对独立的任务和完成任务的步骤和方法,然后逐步完成各个阶段的任务,这有利于软件开发过程的组织和管理,从而降低了整个软件开发过程的困难程度,从而使规模庞大、结构复杂和管理复杂的软件开发变得容易控制和管理。

71.简述软件分析员在系统分析中的任务和作用。

系统分析员（system analyst）是对大型、复杂的信息系统建设任务中，承担分析、设计和领导实施的领军人物。 要做好与客户之间的关系，同时对客户的需求要正确的理解，要选择合适的开发技术，同时做好与客户间沟通交流，学会说服对方。

工作内容：

1、参与用户需求调研、负责系统体系结构、功能、性能的分析和总体设计工作；

2、负责项目的开发流程管理，进行项目的计划、管理、跟进工作；

3、参与并指导开发工程师完成系统详细设计和开发工作，解决相应业务、技术难题；

4、参与编制用户手册、协助客户的系统软件、硬件平台的安装实施工作；

5、制定项目文档格式，编写项目管理规范要求的相关文档

作用：

系统分析工作是解决一个问题的工作，目**标是将一个对计算机应用系统的需求转化成实际的物理实现，**

81.软件工程是如何克服软件危机的？

软件工程是一门应用数学、计算机科学、管理学等多学科知识和方法，以系统化、规范化、可度量的方式开发和维护软件的工程学科。

软件危机是指在软件开发和维护过程中出现的一系列问题，如软件需求不明确、软件设计不合理、软件质量不可靠、软件成本超支、软件进度延误等。

软件工程是为了克服软件危机而产生的，它通过以下几个方面来解决软件危机：

- 软件工程引入了生存期模型，将软件开发和维护过程划分为若干个阶段，每个阶段有明确的目标、输入、输出、活动和标准，以便于对软件过程进行管理和控制。
- 软件工程采用了面向对象的思想，将软件系统视为由对象、类、继承、多态等构成的实体,以便于对软件系统进行抽象和封装,提高软件系统的可重用性和可扩展性。
- 软件工程使用了图形化的工具，如类图、用例图、状态图等，以便于对软件系统进

行建模和编码，提高软件系统的可理解性和可维护性。

- 软件工程实施了质量保证，通过规范的文档、测试、评审等手段，以便于对软件系统进行验证和评价，提高软件系统的可靠性和质量。

82.软件开发主要由哪些方面风险，如何进行软件风险分析？

软件开发的风险是指在软件开发过程中可能出现的不利的或不确定的情况，如需求变更、技术难题、人员流失、资源不足等，它们可能导致软件系统的功能、质量、成本、进度等方面的损失或偏差。

软件风险分析是指对软件开发过程中可能出现的风险进行识别、评估、控制和监测的活动，它主要包括以下几个步骤：

- 风险识别，是指通过收集信息、分析文档、访谈人员等方式，找出软件开发过程中可能存在的风险，并将其记录在风险清单中。
- 风险评估，是指通过定性或定量的方法，对风险清单中的每个风险进行概率和影响的分析，并将其分类为高、中、低等级别。
- 风险控制，是指针对不同级别的风险，制定相应的应对策略，如规避、转移、减轻、接受等，并将其记录在风险计划中。
- 风险监测，是指通过定期的检查、报告、审查等方式，跟踪风险清单和风险计划中的每个风险的状态和变化，并根据实际情况进行调整和更新。

83.程序的编码风格主要体现在哪几个方面？

程序的编码风格即为程序设计风格或编程风格，其主要作用是使无论是程序本人还是其他人，都能比较容易的阅读、理解及修改源代码；

主要表现在四个方面：源程序文档化、数据说明方法，表达式和语句结构及输入/输出方法；

6.软件危机主要有哪些表现?

软件危机的表现有以下几种：

- 对软件开发成本和进度的估计常常很不准确，导致软件项目超支和延期。
- 软件产品的质量往往靠不住，导致软件系统出现故障和漏洞。
- 用户对已完成的软件系统不满意，导致软件系统无法满足用户的需求和期望。
- 软件系统的维护成本高昂，导致软件系统难以适应环境的变化和技术的更新需求增长得不到满足

软件危机是指在软件开发中遇到的问题找不到解决方法，使问题积累起来，形成了尖锐的矛盾。

其表现包括软件开发进度难以控制、软件质量难以保证、软件成本不断提高等。

产生的原因主要是软件规模的扩大和软件复杂性的增加，导致传统的软件开发方法无法满足需求。

96.常见的软件开发模型有那些，各有什么特点？（简述螺旋模型、瀑布模型、演化模型、增量模型各自的特点及它们之间的主要区别）

常见软件开发模型有：**瀑布模型、演化模型、增量模型、螺旋模型、快速原型、V 模型、**

喷泉模型等。

瀑布模型：瀑布模型是一种**经典的软件开发方法**论，它将**软件开发生命周期划分为一系列顺序的阶段**，就像瀑布一样，水流从一个阶段依次流向下一个阶段，每个阶段都有明确的输入和输出。

瀑布模型的特点是：

（1）规定了阶段之间自上而下、相互衔接的固定次序，上一阶段的成果作为下一阶段的输入

（2）每一个阶段完成之后，必须对阶段性成果进行评审，确认之后才可进入下一阶段。

演化模型：演化模型主要针对事先不能完整定义需求的软件项目，此模型可以 减少风险。演化模型**全部系统是增量开发,增量提交**。

增量模型：增量模型在软件开发过程中把软件产品划分为一系列的增量构件,分别进行设计、编程、集成和测试。每个构件由多个相互作用的模块构成,并且能够完成特定的功能。使用增量模型的软件体系结构必须是开放的,加入新构件的过程必须简单方便,新的增量构件不得破坏已经开发出来的产品。因此,**增量模型是一种非整体开发模型,**灵活性好。

快速原型模型：快速原型模型的第一步是建造一个快速原型，实现客户或未来的用户与系统的交互，用户或客户对原型进行评价，进一步细化待开发软件的需求。通过逐步调整原型使其满足客户的要求，开发人员可以确定客户的真正需求是什么；第二步则在第一步的基础上开发客户满意的软件产品。

螺旋模型：螺旋模型，它将**瀑布模型和演化模型结合起来**，强调了其他模型所忽视的风险分析，特别适合于大型复杂的系统。**螺旋模型由风险驱动，强调可选方案和约束条件从而支持软件的重用**，有助于将软件质量作为特殊目标融入产品开发之中。

V 模型：瀑布模型的变体，编码（coding）构成了 V 点，左边是分析和设计，右边是测试和维护，它表达了测试与分析和设计的对应相关，该模型更明确地说明了隐藏在瀑布描绘中的一些迭代和返工。V 模型的特点：

1. **结构清晰**：V 模型将软件测试的各个阶段和开发阶段进行对应，使整个测试过程的结构清晰可见。

2. **并行开展**：V 模型强调测试活动与开发活动的并行进行，以尽早发现和修复软件缺陷，提高软件质量。

3. **强调验证与验证**：V 模型注重对需求的验证和设计的验证，确保软件满足用户需求和设计要求。

4. **易于管理和控制**：V 模型的测试活动与开发活动一一对应，使项目管理和控制更加方便。

喷泉模型：喷泉模型是典型的**面向对象生命周期模型**,是一种**以用户需求为动力,以对象作为驱动的模型**。该模型**克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。“喷泉”一词本身体现了迭代和无间隙特性**。迭代意味着模型中的开发活动常常需要重复多次,在迭代过程中不断地完善软件系统;无间隙是指在开发活动之间不存在明显的边界。**特点：适应变化的迭代开发、增量交付、适合需求不明确与频率变化、适合技术探索型和以客户为中心项目等**。

77.软件工程的文档分哪两大类，主要作用是什么？

软件工程的文档一般分为两大类：

- 软件开发文档，是指在软件开发过程中产生的文档，如需求规约、设计说明、测试计划、用户手册等，它们主要用于描述软件系统的需求、设计、测试、使用等方面的信息，以便于软件开发人员、管理人员、用户等进行沟通和协作。

- 软件维护文档，是指在软件维护过程中产生的文档，如维护请求、维护方案、维护报告等，它们主要用于记录软件系统的修改和更新的原因、方法、结果等信息，以便于软件维护人员、管理人员、用户等进行分析和评价。

78.详细设计的任务是什么？

详细设计是软件工程中软件开发的一个步骤，就是对概要设计的一个细化，就是详细设计每个模块实现算法，所需的局部结构。

详细设计包括业务对象设计、功能逻辑设计、数据库设计和界面设计等工作。详细设计是系统实现的依据，需要考虑所有的设计细节。

详细设计的主要任务是设计每个模块的实现算法、所需的局部数据结构。详细设计的目标有两个：实现模块功能的算法要逻辑上正确和算法描述要简明易懂。

85.如何从需求文档中识别类和对象

从需求文档中识别类和对象的方法有以下几种：

- 词法分析法，是指通过分析需求文档中的名词、动词、形容词等词汇，找出可能是类或对象的候选词，如人、车、订单等是名词，可能是类或对象；学习、驾驶、下单等是动词，可能是方法；年龄、颜色、数量等是形容词，可能是属性。
- 语义分析法，是指通过分析需求文档中的语义关系，找出可能是类或对象的候选词，如具有相同属性和行为的一组实体，可能是类；具有独立存在和标识的实体，可能是对象；具有包含或组成的关系的实体，可能是聚合或组合；具有一般和特殊的关系的实体，可能是继承或泛化。
- 模板分析法，是指通过使用一些预定义的问题模板，对需求文档中的每个候选词进行提问和回答，找出可能是类或对象的候选词，如“它是什么？”、“它能做什么？”、“它有什么？”、“它属于什么？”等。

03、比较原型模型和螺旋模型的优缺点。

原型模型是一个用于验证设计方案或解决方案的工具,可以帮助设计师、开发人员和客户更好地理解和沟通需求和解决方案。
原型模型的优点是快速创建原型，与客户进行交互和反馈，以便更好地理解和确认需 求。它容易适应需求的变化，有利用开发与培训的同步，费用低、开发周期短且对用户 更友好。
原型模型也有一些缺点，客户与开发者对原型理解不同，准确的原型设计比较 困难，不利于开发人员的创新。

螺旋模型则是一种迭代和风险驱动的软件开发模型，它将软件开发分为四个阶段：计 划、风险分析、工程实现和评审。在每个迭代中，开发从员需要根据风险管理计划来确 定下一步的工作。
螺旋模型的优点是灵活性高，可以在项目的各个阶段进行变更 ，客 户始终参与每个阶段的开发，保证了项目不偏离正确方法以及项目的可按性。
螺旋模型的缺点：它 需要具有相当丰富的风险评估经验和专门知识，在风险较大的项目开发中，如果未能及 时标识风险，势必造成重大损失；会增加开发成本，延迟提交时间。

115、软件质量控制与软件质量保证之间有何区别?

①软件质量保证：软件质量保证是一种预防性的活动，旨在确保软件开发过程中遵循适当的流程、标准和规范，以提高最终软件产品的质量。
质量保证关注的是整个软件开发过程中的质量管理，包括规划、组织、实施和监控各项活动，以确保软件开发团队按照正确的方式进行工作，从而最大程度地降低缺陷和问题的出现。
②软件质量控制：软件质量控制是一种检测性的活动，旨在检查和验证软件产品是否符合预定的质量标准和要求。
质量控制主要关注的是已经开发完成的软件产品，通过执行测试、审查和检查等活动，发现和修复软件中存在的缺陷和问题，以确保软件产品的质量符合预期。

质量保证与质量控制的区别

软件质量保证与质量控制的区别如下表所示 ：

项目	质量保证（QA）	质量控制(QC)
定义	质量保证是一组活动，可确保始终保持软件开发过程中使用的过程质量。	QC是一组用于检测已开发软件中的缺陷的活动。
关注重点	QA的重点是通过关注流程来防止开发软件中的缺陷。	QC的重点是通过关注测试过程来识别开发软件中的缺陷。
如何做	建立高质量的管理系统，并定期审核开发软件的操作是否符合要求。	通过使用开发软件中的测试技术和工具来检测和消除质量问题元素。
为什么做	质量保证通过使用包括文档在内的系统活动来确保质量问题的预防。	QC通过使用流程和技术来实现和维护高质量的软件，从而确保识别和消除缺陷。
面向	质量保证是面向流程的。	QC是面向产品的。
过程类型	质量保证是一个积极主动的过程。它关注改善开发;在测试期间不会出现缺陷。	QC是一种反应过程，因为它涉及在产品开发之后和产品发布之前识别缺陷。
责任	开发团队的每个成员都负责QA	只有特定的测试团队负责QC
示例	验证软件流程。	检验软件功能和流程。

21、业务需求、用户需求、功能需求之间有何区别?

软件需求包括三个不同的层次——业务需求、用户需求和功能需求，当然也包括也包括非功能需求。
业务需求：描述组织或客户高层次的目标。业务需求就是系统目标，它必须是业务导向、可度量、合理、可行的。业务需求是需求定义的产物；
用户需求：描述的是用户的目标，或用户要求系统必须能完成的任务。用户需求是需求捕获的结果；
功能需求：规定开发人员必须在产品中实现的软件功能，用户利用这些功能来完成任务，满足业务需求。
三者之间区别：
（1）业务需求：公司想如何解决用户的问题，如何满足用户的欲望，并将利益最大化。
（2）用户需求。用户需要在应用系统中实现什么东西，为实现这个目标，需要用户提供的全部的详细的业务说明，业务流程，表格样式等。
（3）功能需求。将用户需求归类分解为计算机可以实现的子系统和功能模块，用设计语言描述和解释用户的需求，以达到可以指导程序设计的目的。

48.简述总体设计的一般过程？

总体设计又称为概要设计或初步设计。总体设计过程通常由两个主要阶段组成：

系统设计:确定系统的具体实现方案。结构设计:确定软件结构。

9 个步骤：

设想供选择的方案；选取合理的方案；推荐最佳方案；功能分解；设计软件结构；设计数据库；制定测试计划；书写文档；审查和复审

3.什么是面向数据流的设计，数据流包括哪几种？

面向数据流的设计方法把**信息流映射成软件结构**，信息流的类型决定了映射的方法，**根据系统的数据流进行的设计，又称结构化设计**

数据流包括：变换流、事务流

1．变换流

根据基本系统模型，信息通常以外部世界的形式进入软件系统，经过处理以后再以外部世界的形式离开系统。

2．事务流

基本系统模型意味着变换流，因此，原则上所有信息流都可以归结为这一类。但是当数据流是以事务为中心的，也就是说，数据沿输入通路到达一个处理 T，这个处理根据输入数据的类型在若干个动作序列中选出一个来执行。这类数据流应该划为一类特殊的数据流，称为事务流。

8.怎样完成从数据流图到程序结构图的转换？

从以下 4 个步骤进行:

1.重画数据流图 .

2.区分有效（逻辑）输入、有效（逻辑）输出和**中心变换部分**

3.进行一级分解，设计上层模块

4.进行二级分解，设计中、下层模块

13.面向数据流的设计方法包含那些步骤？

1）确定数据流的类型：变换流还是事务流

2）划定流界

3）将数据流图转换为软件结构

4）通过设计复审和启发式策略精化所得到软件结构

18.画数据流图原则是什么？

（1）明确系统界面。

（2）自顶向下逐层扩展。

（3）合理布局。

（4）数据流图绘制过程，就是系统的逻辑模型的形成过程，必须始终与用户密切接触，详细讨论，不断修改，也要和其他系统建设者共同商讨一求一致意见。

23.数据流图作用是什么？

DFD 是**描述数据处理的工具**。 数据流图从数据传递和加工的角度，以图形的方式刻画数据流从输入到输出的移动变换过程。从而便于需求分析.

28.简述数据字典的作用

数据字典是存储有关数据的来源、说明、与其他数据的关系、用途和格式等信息的数据库。**数据字典的作用是：**

- 提供系统的逻辑模型，与数据流图配合使用。
- 规范数据的定义和命名，保证数据的一致性和完整性。
- 帮助开发人员和用户了解系统的数据结构和流程。
- 方便管理和维护下拉框等表单元素的选项。

38.什么是结构化分析，有什么特点？

是面向数据流进行需求的分析，遵循自顶向下、逐步求精原则。

主要特点是快速、自然和方便。成功率较高，发展较为成熟；简单、易掌握，适应于瀑布模型；特别适合于数据处理领域中的应用，对规模大的项目，特别复杂的应用不太适应。难于解**决软件重用问题**，难于适应需求的变化。

68.简述层次方框图与软件结构图的异同点。

相同：形式类似。

不同：

1,层次图描绘软件的**层次结构.层次方框图描绘的是数据结构**。

2,层次图的**方框**表示模块或子模块。层次方框图的方框表示数据结构整体或其子集。

3,层次图的**连线**表示**调用关系**，层次方框图表**示组成关系**。

73.事务型软件结构图有什么特点，原因是什么？

事务型软件结构图是一种软件结构图，它描述了软件系统中的事务处理过程，即软件系统如何响应用户或外部事件的输入，执行相应的业务逻辑，产生相应的输出或反馈。

事务型软件结构图有以下几个特点：

- 事务型软件结构图是一种自顶向下的结构图，它从整体到局部，逐层展示了软件系统的功能结构和数据流。
- 事务型软件结构图是一种动态的结构图，它强调了软件系统的行为和状态变化，而不是静态的组成和关系。
- 事务型软件结构图是一种抽象的结构图，它忽略了软件系统的具体实现细节，只关注软件系统的功能和数据。

事务型软件结构图之所以具有这些特点，主要是因为：

事务型软件系统通常是面向用户或外部事件的软件系统，它需要提供清晰和完整的功能视图和数据视图，以便于用户或外部事件的理解和使用。

事务型软件系统通常是动态和复杂的软件系统，它需要表达软件系统的运行逻辑和状态转换，以便于软件系统的分析和设计。

事务型软件系统通常是高层次和跨平台的软件系统，它需要屏蔽软件系统的实现细节和平台差异，以便于软件系统的移植和维护。

88.好的软件体系结构设计应遵循哪些原则？

软件体系结构设计的原则有以下几种：

- 抽象原则，是指将软件系统分解为若干个抽象的组件和连接器，以便于对软件系统进行高层次的描述和理解。
- 模块化原则，是指将软件系统划分为若干个相对独立的模块，以便于对软件系统进行分工和协作。
- 封装原则，是指将软件系统中的每个组件或模块的内部细节隐藏起来，只暴露出必要的接口和协议，以便于对软件系统进行保护和隔离。
- 开放原则，是指将软件系统设计成可以与其他软件系统进行交互和集成，以便于对软件系统进行扩展和复用。
- 低耦合高内聚原则，是指将软件系统中的组件或模块之间的依赖关系尽量减少，而将组件或模块内部的相关性尽量增强，以便于对软件系统进行修改和测试。

108、结构化分析和结构化设计的鸿沟。

结构化分析和结构化设计是一种面向数据流的传统软件开发方法。分析阶段的主要模型是数据流程图，设计阶段的主要模型是软件模块结构图，数据流程图和软件模块结构图之间需要进行转换。不同的人转换出的模块图可能不同，有很大的随意性。在结构化分析中，模块之间的关系被局限于信息流，这限制了对模块之间众多关系的表达，也无法体现模块和模块之间其他的众多关系，包含各种各样的结构、行为、依赖、包含、继承、关联关系等等。而面向对象编程则能够有效解决这些问题。

110、变换型数据流图和事务型数据流图的区别

变换型数据流图和事务型数据流图是两种不同的数据流图类型。

变换型数据流图是一种线性结构，由输入、变换和输出三部分组成，事务型数据流图则呈束状，即一束数据流平行流入或流出，可能同时有几个事务要求处理。

事务型数据流图的特点是将它的输入流分离成许多发散的数据流，形成许多加工路径，并根

据输入的值选择其中一个路径来执行。
这两种典型的结构分别可以通过“变换分析”和“事务分析”技术，导出相应的初始模块结构图。
变换分析和事务分析技术的思想是首先设计出顶层模块，然后自顶向下、逐步求精，最后得到一个满足数据流图所表示的系统模块初始结构图，即系统的物理模型。

112、变换型和事务型数据流图各有什么特点？

变换型数据流图和事务型数据流图是两种不同的数据流图类型。
变换型数据流图是一种线性结构，由输入、变换和输出三部分组成，事务型数据流图则呈束状，即一束数据流平行流入或流出，可能同时有几个事务要求处理。
事务型数据流图的特点是将它的输入流分离成许多发散的数据流，形成许多加工路径，并根据输入的值选择其中一个路径来执行。

123、简述数据流图(DFD)与流程图(Flow Chart)的异同点。

数据流图(DFD)与流程图(Flow Chart)的相同：两者都是描述数据的流动情况。
数据流图(DFD)与流程图(Flow Chart)区别：
（1）数据流图（date flow diagram ， DFD)，是SA方法中用于表示系统逻辑模型的一种工具，它以图形的方式描绘数据在系统中流动和处理的过程，由于它只反映系统必须完成的逻辑功能，所以它是一种功能模型，是从数据的角度来描述一个系统的；而流程图则是从对数据加工的角度来描述系统的；
（2）数据流图中的箭头是数据流，而流程图中的箭头则是控制流，它表达的是程序执行的次序；
（3）数据流图适合于宏观地分析一个组织业务概况，而程序流程图只适合于描述系统中某个加工的执行细节。
（4）数据流程图应该重点描述了数据加工的过程，主要是模块内部，数据流图则是描述模块之间的关系。

119、软件的概要设计与软件的详细设计之间有什么联系和区别?

概要设计和详细设计是软件开发中两个非常重要的步骤。
概要设计是指在软件开发过程中，对软件的整体结构、组件、模块和交互进行规划和设计的过程。
详细设计是在**概要设计阶段之后的一阶段**，是对概要设计中所确定的结构的具体实现。
概要设计和详细设计两者之间的区别在于粒度的不同。
概要设计主要是指对系统整体的规划，主要关注整个系统的结构和模块关系；
详细设计主要是对概要设计中所确定的系统结构进行更为具体的实现，主要关注系统具体实现的内容，涉及到实际编码细节和算法实现等。
两者之间的联系：
在于概要设计是详细设计的基础，概要设计中所规划的结构和模块关系就是为详细设计所服务的；
而详细设计则需要依靠概要设计中的设计思路和规划，合理地实现整个系统的功能。

5.什么是对象，对象有哪几种形式？

对象是面向对象编程中的基本概念,它是对现实世界中的实体、事物或概念的抽象表示。对象具有以下几种形式：

类：类是对象的模板，它定义了对象的属性和方法，但并不占用实际的空间。

实例：实例是类的具体化，它根据类的定义创建了对象，并占用实际的空间。

属性：属性是对象的特征，它描述了对象的状态或数据。

方法：方法是对象的行为，它描述了对象可以做什么或如何改变自己的状态。

链：链是对象之间的关系，它描述了对象如何相互影响或通信。

10.什么是类，类与对象间是什么关系？

类是对一类事物的抽象描述，它定义了事物的属性和方法，但并不占用实际的空间。对象是类的具体化，它根据类的定义创建了事物，并占用实际的空间。类与对象之间的关系可以总结为以下几点：

类是对象的模板，对象是类的实例。例如，人是一个类，小韩、小石等是人的对象。

类描述了对象的共同特征，对象表示了类的个体差异。例如，人有名字、性别、年龄等属性，小韩、小石等有不同的名字、性别、年龄等值。

类可以创建多个对象，对象只属于一个类。例如，人可以创建小韩、小石等多个对象，小韩、小石等只属于人这个类。

15.面向对象方法特征有哪些？

封装性、继承性、多态性、抽象

封装。把数据和方法封装在内部，构成一个整体功能，只提供某个方法给外面访问。

继承。继承是解决代码的复用，是类和类之间的关系，使得子类具备父类中得方法。

多态。多态是指同一个方法在不同的对象上有不同的表现形式或行为。

抽象。抽象是将有关事物的共性归纳、集中的过程，忽略不重要的细节。

25.面向对象设计涉及那几个主要活动？

分析活动、设计活动、编码活动

30.面向对象设计如何体现抽象信息、隐藏和模块化这三个概念的？

通过类的引入，封装,把数据结构和操作这些数据的方法紧密地结合在一起所构成的模块.对象的封装性实现隐藏。

35.举例说明什么是类的整体部分结构？*

如建立一个时钟类，这个类中的属性如秒、分钟、小时等，加上对属性的操作如秒记时，分钟记时，小时记时等，就是一个类的整体部分结构。

0.举例说明什么是类的组装结构关系

如建立一个时钟类，这个类中的属性如秒、分钟、小时等，加上对属性的操作如秒记时，分钟记时，小时记时等，就构成一个类的组装结构关系。

50.类的开发有几个途径，如何进行类的开发？

一是新建基类，二是通过类间的继承来进行类的开发。

75.Coad/Yourdon 的 OOA 模型包括那几个方面？

Coad 和 Yourdon 方法可分为面向对象分析（OOA）和面向对象设计（OOD）两部分。在 OOA 中，**建立了概念模型，由类与对象、属性、服务、结构和主题等 5 个分析层次组成**。

（1）类与对象。从问题域、文字出发，寻找并且标识类与对象。

（2）属性。确定对象信息及其之间的关系。可分为原子概念层的单个数据，和类结构中的公有属性与特定属性。

（3）服务。标识消息连接和所有服务说明。

（4）结构。标识类层次结构，确定类之间的整体部分结构与通用特定结构。

（5）主题。主题是比结构更高层次的模块，它与相关类在一起，控制着系统的复杂度。

在 Coad 和 Yourdon 方法的系统设计阶段也采用了 OOA 阶段的 5 个层次来描述以下设计模型的 4 个部分。

（1）问题域。

（2）用户界面。包括问题域与用户、系统外部和专用设备、磁盘文件和数据管理界面等。

（3）任务管理。包括任务的运行和任务的交互管理等。

（4）数据库管理。包括文件管理和数据库管理等。

76.面向对象生存期模型与传统的生存期模型有什么区别?

存期模型是一种描述软件开发过程的方法，它将软件开发过程划分为若干个阶段，每个阶段有明确的目标、输入、输出、活动和标准。

面向对象生存期模型是一种基于面向对象思想的生存期模型，它强调软件系统的对象、类、继承、多态等概念，以及软件系统的可重用性和可扩展性。

传统生存期模型是一种基于结构化思想的生存期模型，它强调软件系统的功能、数据、流程等概念，以及软件系统的可靠性和可维护性。

面向对象生存期模型与传统生存期模型有以下几个区别：

- 面向对象生存期模型更加灵活和迭代，它允许在任何阶段进行修改和反馈，而传统生存期模型更加严格和线性，它要求按照固定的顺序进行开发。
- 面向对象生存期模型更加注重软件系统的设计和实现，它使用类图、用例图、状态图等图形化的工具进行建模和编码，而传统生存期模型更加注重软件系统的分析和测试，它使用数据流图、判定表、判定树等符号化的工具进行规约和验证。

- 面向对象生存期模型更加适合开发复杂和动态的软件系统，它可以利用对象、类、继承、多态等机制进行抽象和封装，提高软件系统的可重用性和可扩展性，而传统生存期模型更加适合开发简单和稳定的软件系统，它可以利用功能、数据、流程等机制进行分解和组合，提高软件系统的可靠性和可维护性。

80.什么是对象关系模型和对象行为模型，有什么不同？

对象关系模型和对象行为模型是两种面向对象的建模方法，它们分别用于描述软件系统中的对象之间的静态结构和动态行为。

对象关系模型是一种用于描述软件系统中的对象之间的静态结构的模型，它主要包括以下几个方面：

- 对象，是指软件系统中具有一定属性和行为的实体，如人、车、订单等。
- 类，是指具有相同属性和行为的一组对象的抽象，如学生类、汽车类、订单类等。
- 属性，是指对象或类所具有的特征或状态，如姓名、颜色、数量等。
- 方法，是指对象或类所具有的行为或功能，如学习、驾驶、下单等。
- 关系，是指对象或类之间的联系或依赖，如继承、关联、聚合等。

对象关系模型通常使用类图来表示，类图是一种图形化的工具，它用矩形框表示类，用连线表示关系，用文字表示属性和方法。

对象行为模型是一种用于描述软件系统中的对象之间的动态行为的模型，它主要包括以下几个方面：

- 事件，是指软件系统中发生的事情或变化，如按键、点击、输入等。事件是某个特定时刻所发生的事情，它是对引起对象从一种状态转换到另一种状态的现实世界中的事件的抽象。
- 消息，是指软件系统中对象之间传递的信息或请求，如调用方法、返回值等。
- 状态，是指软件系统中对象或类在某一时刻的情况或条件，如开启、关闭、就绪等。
- 活动，是指软件系统中对象或类在某一状态下执行的操作或动作，如初始化、计算、输出等。
- 转移，是指软件系统中对象或类从一个状态转变到另一个状态的过程或条件，如满足条件、触发事件等。

对象行为模型通常使用用例图、状态图、活动图等来表示，它们都是一种图形化的工具，它们用圆形框表示事件或状态，用箭头表示消息或转移，用文字表示活动或条件。

55.什么是 OOA 建模语言，它应该包括哪些方面？

OOA 建模语言是指用于进行面向对象分析（Object-Oriented Analysis）的语言，它可以描述系统的功能、结构、行为和约束等方面。OOA 建模语言应该包括以下方面：

- 类图：用于描述系统中的类及其属性、方法和关系，如泛化、实现、关联、聚合、组合和依赖等。
- 用例图：用于描述系统的功能需求，以及参与者和用例之间的关系，如包含、扩展和泛化等。
- 状态图：用于描述系统中某个对象的状态变化和触发条件，以及状态之间的转移关系。
- 活动图：用于描述系统中的业务流程或算法，以及活动之间的控制和数据流关系。

- 序列图：用于描述系统中对象之间的交互过程，以及消息的发送和接收顺序。
- 协作图：用于描述系统中对象之间的交互过程，以及对象的结构和位置关系。
- 组件图：用于描述系统中的组件及其接口和依赖关系，以及组件之间的通信方式。
- 部署图：用于描述系统中的物理结构，包括节点、设备、连接器和部署规范等。

60.简述 RUP（Rational Unified Process）的基本内容

RUP（Rational Unified Process）是一种面向对象且基于网络的程序开发方法论，由 Rational 公司提出，是一种软件开发过程框架，基于面向对象符号体系给出了有关软件开发过程组织及实施的指导

迭代式开发。RUP 主张在软件开发的早期阶段就想完全、准确的捕获用户的需求几乎是不可能的，因此采用迭代式开发，将软件生命周期分解为多个迭代周期，每个迭代周期都包含需求分析、设计、实现和测试等活动。

用例驱动。RUP 以用例为核心，用用例来描述系统的功能需求，用用例来驱动系统的分析、设计、实现和测试。

以架构为中心。RUP 认为系统架构是软件开发过程中最重要的工件，它决定了系统的质量属性和可维护性，因此要在早期阶段就确定系统架构，并在后续阶段不断地验证和完善。

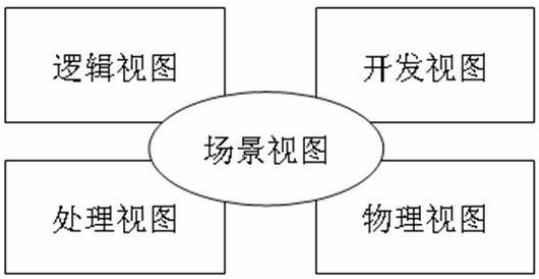
可配置。RUP 是一个通用的过程模板，包含了很多开发指南、制品、开发过程所涉及到的角色说明，由于它非常庞大所以对具体的开发机构和项目，用 RUP 时还要做裁剪，也就是要对 RUP 进行配置。

70.什么是 RUP（Rational Unified Process）的 4+1 视图？

RUP（Rational Unified Process，统一软件开发过程，统一软件过程是一个面向对象且基于网络的程序开发方法论。

要开发出用户满意的软件并不是件容易的事，软件架构师必须全面把握各种各样的需求、权衡需求之间有可能的矛盾之处，从不同视图进行架构设计，来分门别类地将不同需求一一满足

Philippe Kruchten 提出的 4+1 视图方法



该方法的不同架构视图承载不同的架构设计决策，支持不同的目标和用途：

逻辑视图：当采用面向对象的设计方法时，逻辑视图即对象模型。对象模型表示了静态的、 结构化的系统数据性质，描述了系统的静态结构，它是从客观世界实体的对象关系角度来描述,表现了对象的相互关系。

开发视图：描述软件在开发环境下的静态组织。

处理视图：描述系统的并发和同步方面的设计。

物理视图：描述软件如何映射到硬件，反映系统在分布方面的设计。

90.UML 中有哪些动态建模的工具，各表达什么内容？

UML 中有以下几种动态建模的工具，各表达如下内容：

- 用例图，是指描述软件系统与外部参与者之间的交互和功能需求的图。
- 时序图，是指描述软件系统中对象之间按时间顺序进行的消息交换的图。
- 协作图，是指描述软件系统中对象之间的结构和消息交换的图。
- 状态图，是指描述软件系统中对象或类在不同状态下的行为和转换的图。
- 活动图，是指描述软件系统中的业务流程或算法逻辑的图。

65.什么是 Use Cases Model，主要作用是什么？

是系统既定功能及系统环境的模型

用例模型最重要的作用是将系统行为传达给客户或最终用户。

95.UML 中的接口类有什么特点，举例说明。

接口描述了一组实现这一接口的分类器所必须共同遵守的合同，这种合同包括结构上的也包括行为上的，此外，接口是不能实例化的。接口实现则是一种依赖关系，其描述了接口与分类器之间的一种特殊的实现关系。

图 1 示例了接口的棒棒糖表示方法，其中“棒棒糖”表示与之相连的类提相应的接口，也就是说相关联的类实现了这一接口。

从图中我们可以看出，ProximitySensor 类实现了 ISensor 接口，而 TheftAlarm 类则需要一个 ISensor 接口。由于两个类一个提供，而另一个需要，因此我们可以将其连接在一起，如图 2 所示。



图 1 接口的棒棒糖表示法



图 2 接供和需要接口的图形可以连接在一起

100.简要说明 RUP 中阶段、核心工作流和迭代的关系。

RUP（Rational Unified Process，统一软件开发过程，统一软件过程)是一个面向对象且基于网络的程序开发方法论。

RUP 中的软件生命周期在时间上被分解为四个顺序的阶段，分别是：初始阶段（Inception）、细化阶段（Elaboration）、构造阶段（Construction）和交付阶段（Transition)。每个阶段结束于一个主要的里程碑（Major Milestones)；每个阶段本质上是两个里程碑之间的时间跨度。在每个阶段的结尾执行一次评估以确定这个阶段的目标是否已经满足。如果评估结果令人满意的话，可以允许项目进入下一个阶段。

RUP 中有 9 个核心工作流，分为 6 个核心过程工作流（Core Process Workflows）和 3

个核心支持工作流（Core Supporting Workflows）。尽管 6 个核心过程工作流可能使人想起传统瀑布模型中的几个阶段，**但应注意迭代过程中的阶段是完全不同的，这些工作流在整个生命周期中一次又一次被访问。9 个核心工作流在项目中轮流被使用，在每一次迭代中以不同的重点和强度重复。**

4.什么时候软件测试结束，为什么？

软件测试的结束标准通常是根据项目的需求和目标来确定的。以下是一些通用的测试结束标准：

1.所有测试用例都已执行：确保所有计划的测试用例都已被执行，包括功能测试、性能测试、安全测试等各种类型的测试。

2.所有缺陷已修复：所有已识别的缺陷都已被记录、分类和追踪，并且经过验证的已经被修复。

3.测试覆盖率满足要求：根据项目的要求，确保测试覆盖了系统的各个方面，包括功能、性能、安全等。

4.性能满足要求：如果项目有性能要求，确保系统在负载和性能方面达到了这些要求。

5.安全测试通过：如果项目涉及安全性要求，确保系统在安全性方面通过了测试，并没有明显的漏洞或弱点。

6.用户验收测试通过：如果有用户验收测试，确保系统满足用户的期望和需求。

7.测试报告和文档已准备完毕：确保所有必要的测试文档，包括测试计划、测试用例、测试报告等已经准备完毕。

8.测试团队和利益相关者已确认：确保测试团队和利益相关者都已经确认测试的结束，并且对测试结果满意。

9.上线准备就绪：确保系统已准备好上线，包括备份和恢复计划、数据迁移、培训和支持等。

10.风险评估已完成：对项目的风险进行评估，确保已经采取措施来降低或管理这些风险。

14.什么叫测试用例，如何设计测试用例

测试用例（Test Case）是为某个特殊目标而编制的一组测**试输入、执行条件以及预期结果，**以便测试某个程序路径或核实是否满足某个特定需求。

测试用例的设计方法有多种，比如：

等价类划分法。把输入数据或操作条件分成有效等价类和无效等价类，从每个等价类中选取代表性的值作为测试数据。

边界值分析法。在等价类划分的基础上，针对输入数据或操作条件的边界值或极限值进行测试。

错误推测法。根据经验或直觉，推测可能出现错误的地方，设计相应的测试用例。

判定表法。当一个界面中有多个控件，不同控件的组合会得到不同的结果时，用表格的形式列出所有可能的组合和对应的结果。

因果图法。当输入条件或输出条件的组合比较多时，用图形的方式分析判定表，简化测试用例的设计。

正交实验法。当输入条件或输出条件有多个因素和水平时，用数学概率学知识，设计最经济的实验路径。

场景法。模拟用户操作场景，测试多个功能组合。

流程图法。用流程图的方式展现基于用户使用场景的测试步骤。

19.什么叫基本路径测试，如何确定基本路径？

实际程序中路径是个很庞大的数字，所有路径覆盖是不现实的，测试中把路径数压缩在一定范围内，称为基本路径测试。

在程序控制流图的基础上，通过分析控制构造的环路复杂性测度，导出基本可执行路径的基本集合。

20.简述 USE CASE 的作用。

测试用例：

在不展现一个系统或子系统内部结构的情况下，对系统或子系统的某个连贯的功能单元的定义和描述。其实 Use Case 就是对系统功能的描述而已，不过一个 Use Case 描述的是整个系统功能的一部分，这一部分一定要是在逻辑上相对完整的功能流程。

测试用例是为某个特殊目标而编制的一组测试输入、执行条件以及预期结果，以便测试某个程序路径或核实是否满足某个特定需求。

测试用例的作用：

1，通过一个用例来证明被测软件的某功能符合需求说明书中规定的要求，可以通过设计正反两方面的测试用例来验证

2，可以保证一个软件被测试的有效性，使测试人员知道哪些些功能以被测，哪些功能还需要测试，从而避免漏测，重复测，提高测试效率

3，指导测试的工作，保证测试是有计划的实施，而不是随意性的测试

4，为公司留下财富，为后期软件维护提高帮助，为公司新人进来提供指导，在测试的时候可以尽量把人为因素的影响减少到最小。保障软件测试质量的稳定

5，可以做为评估测试结果的，为编写测试报告提供依据。

6，分析缺陷的标准，通过收集缺陷，对比测试用例和缺陷数据库，分析确证是漏测还是缺陷复现。漏测反映了测试用例的不完善，应立即补充相应测试用例，最终达到逐步完善软件质量。而已有相应测试用例，则反映实施测试或变更处理存在问题。

24.简述回溯测试和作用。

回溯法采用试错的思想，它尝试分步的去解决一个问题。在分步解决问题的过程中，当它通过尝试发现现有的分步答案不能得到有效的正确的解答的时候，它将取消上一步甚至是上几步的计算，再通过其它的可能的分步解答再次尝试寻找问题的答案。回溯法通常用最简单的递归方法来实现，在反复重复上述的步骤后可能出现两种情况：

1、找到一个可能存在的正确的答案

2、在尝试了所有可能的分步方法后宣告该问题没有答案

作用：

1、找出满足约束条件的所有解。

34.什么是因果图，用于何处？*

因果图是一种表示因果关系的图形工具，用于分析和解决问题。
因果图是根据分析时对各种条件可能产生的结果的一种对照分析法。
因果图方法是根据 输入与输出之间的因果关系来设计用例的。
其测试用例的基本依据可以是分析阶段的决策表
根据因果关系建立因果关系图
建立决策表获得用例

39.什么是边界值分析，与等价类划分有什么不同？

边界值分析法就是对输入或输出的边界值进行测试的一种黑盒测试方法。
（1）、边界值分析不是从某等价类中随便挑一个作为代表，而是使这个等价类的每个边界都要作为测试条件。
（2）、边界值分析不仅考虑输入条件，还要考虑输出空间产生的测试情况。

44.简述在确定等价划分测试用例设计中遵循的原则。

等价划分测试用例设计是一种软件测试方法，它将软件的输入或输出域划分为若干个等价类，每个等价类代表了相同的功能或错误行为。然后从每个等价类中选取一个或几个代表性的值作为测试用例，以此来减少测试用例的数量，提高测试效率。
等价划分测试用例设计遵循以下原则：

- 有效等价类原则：有效等价类是指符合软件规格说明的输入或输出域，从有效等价类中选取的测试用例应该使软件正常运行。
- 无效等价类原则：无效等价类是指不符合软件规格说明的输入或输出域，从无效等价类中选取的测试用例应该使软件出现错误或异常。
- 单一故障假设原则：单一故障假设是指在一个测试用例中只包含一个错误，这样可以更容易地定位和修复错误。

49.什么是软件测试中逻辑覆盖，一般软件测试至少应保证哪些覆盖？

是以程序内部的逻辑结构为基础的测试技术，是通过对程序逻辑结构的遍历实现程序的覆盖，这一方法要求测试人员对程序的逻辑结构有清楚的了解。至少应保证语句覆盖。

54.为什么说在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价？

在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价，是因为这样会导致以下问题：

- **需求变更：**如果需求理解错误，就需要重新修改需求文档，重新评审需求，重新制定开发计划，重新分配开发任务，重新编写代码，重新测试代码，重新部署上线。这些工作都会消耗大量的时间、人力和资源。
- **质量风险：**如果需求理解错误，就可能导致开发出来的产品不符合客户的期望和需求，或者存在严重的缺陷和漏洞。这些问题会影响产品的质量、性能、安全性和用户体验，甚至可能造成客户的损失和投诉。
- **信誉损失：**如果需求理解错误，就可能导致产品不能按时交付，或者交付的产品不

满足客户的要求。这些情况会损害开发团队和公司的信誉和形象，降低客户的满意度和忠诚度，影响未来的合作机会。

因此，在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价。所以，在开发过程中，应该尽量避免需求理解错误，通过有效的沟通、评审、文档、原型等方式，保证测试人员、开发人员和客户对需求的理解一致。

64.单元测试时，为什么需要开发驱动模块和桩模块？

这是因为在开发设计的初级阶段，完成一个单元的开发任务后，这个单元相关联上面、下面的模块尚未开发出来，为了要测试所开发单元，所以要驱动模块和桩模块

69.简述黑盒子测试和白盒子测试及其适用性。

黑盒子测试是一种把程序看做一个黑盒子，完全不考虑程序内部结构和处理过程的测试方法。黑盒子测试是对程序的接口进行测试，它只检验程序功能是否能够要找预定的那样正常的工作。

黑盒子测试力图发现以下问题：

1）功能是否不正确或者不符合要求

2）界面是否错误

3）数据结构错误或外部数据库访问错误

4）性能不符合

白盒测试也称结构测试或逻辑驱动测试，它是按照程序内部的结构测试程序，通过测试来检测产品内部动作是否按照设计规格说明书的规定正常进行，检验程序中的每条通路是否都能按预定要求正确工作。 这一方法是把测试对象看作一个打开的盒子，测试人员依据程序内部逻辑结构相关信息，设计或选择测试用例，对程序所有逻辑路径进行测试，通过在不同点检查程序的状态，确定实际的状态是否与预期的状态一致。

对于白盒子和黑盒子测试，他们两者是不能互相替代的，白盒子测试主要是早阶段进行的，黑盒子测试时后阶段进行的。

74.单元测试、组装测试和确认测试各自主要的目标是什么，相互有什么关系？

单元测试（unit testing），是指对软件中的最小可测试单元进行检查和验证。

组装测试（也叫集成测试，联合测试）是单元测试的逻辑扩展。在单元测试的基础上，将所有模块按照设计要求（如根据结构图）组装成为子系统或系统，进行集成测试。

确认测试的目的是向未来的用户表明系统能够像预定要求那样工作。经集成测试后，已经按照设计把所有的模块组装成一个完整的软件系统，接口错误也已经基本排除了，接着就应该进一步验证软件的有效性，这就是确认测试的任务，即软件的功能和性能如同用户所合理期待的那样。

单元测试是在开发时对小的模块的功能进行测试，集成测试是在几个模块集成的时候对模块之间的接口、集成之后的模块的功能进行测试。这两个测试都需要对软件内部结构比较熟悉，所以通常使用白盒测试的方法。

而确认测试的目的在于测试软件产品是否符合最初定义的用户需求，它不关心内部结果，只关心能否实现功能，也就是相当于把软件当做一个黑盒子，看输入数据能否有正确的输出。

79.为什么把软件测试的目标定义为只是发现错误？

软件测试的目标是发现错误，是因为：

- 软件测试是一种验证软件系统是否符合需求和设计的活动，它通过执行软件系统或其部分，观察软件系统的输出或行为，与预期的输出或行为进行比较，从而发现软件系统中存在的错误或缺陷。
- 软件测试不能保证软件系统是完全正确或完全满足用户需求的，它只能证明软件系统存在错误或不满足用户需求，而不能证明软件系统不存在错误或完全满足用户需求。因此，软件测试的目标不是证明软件系统的正确性或质量，而是发现软件系统的错误或问题。
- 软件测试是一种有限的活动，它受到时间、资源、技术等因素的限制，它不能覆盖所有可能的输入、输出、状态、环境等情况，它只能选择一部分有代表性或重要性的测试用例进行测试。因此，软件测试的目标不是检查软件系统的所有方面，而是尽可能多地发现软件系统的重要或严重的错误或缺陷。

软件测试是对软件计划、软件设计和软件编码进行纠错的活动,测试的目的是找出整个软件开发周期中各个阶段的错误,分析错误的性质和位置并加以纠正。纠正的过程包括对文档和代码的修改,找错的活动被称为测试,而纠错的过程被称为调试。

(1)测试是为了发现程序中的错误而执行程序的过程。

(2)好的测试方案是极有可能发现迄今尚未发现的尽可能多的错误的测试。

(3)成功的测试是发现了迄今尚未发现的错误的测试。

84.简述软件测试中桩模块和驱动模块的作用。

桩模块（Stub）是指模拟被测试的模块所调用的模块，而不是软件产品的组成的部分。主模块作为驱动模块,与之直接相连的模块用桩模块代替。

桩模块的作用：在集成测试前要为被测模块编制一些模拟其下级模块功能的“替身”模块，以代替被测模块的接口，接受或传递被测模块的数据，这些专供测试用的“假”模块称为被测模块的桩模块。

传统的单元测试术语（unit testing terminology），包括了驱动模块（driver） 和桩模块（stub）。驱动模块是用来模拟被测试模块的上一级模块，相当于被测模块的主程序。它接收数据，将相关数据传送给被测模块，启用被测模块，并打印出相应的结果。

作用：驱动模块的目的很单纯，就是为了访问类库的属性和方法，来检测类库的功能是否正确；

94.软件测试有几种方法，每种方法的特点是什么？

1、按是否查看程序内部结构分为：

（1）、黑盒测试（black-box testing）：只关心输入和输出的结果

（2）、白盒测试（white-box testing）：去研究里面的源代码和程序结构

2、按是否运行程序分为：

（1）、静态测试（static testing）：是指不实际运行被测软件，而只是静态地检查程

序代码、界面或文档可能存在的错误的过程。

（2）、动态测试（dynamic testing），是指实际运行被测程序，输入相应的测试数据，检查输出结果和预期结果是否相符的过程

3、按阶段划分：

（1）单元测试（unit testing），是指对软件中的最小可测试单元进行检查和验证。

（2）集成测试（integration testing），是单元测试的下一阶段，是指将通过测试的单元模块组装成系统或子系统，再进行测试，重点测试不同模块的接口部门。集成测试就是用来检查各个单元模块结合到一起能否协同配合，正常运行。

（3）系统测试（system testing），指的是将整个软件系统看做一个整体进行测试，包括对功能、性能，以及软件所运行的软硬件环境进行测试。系统测试的主要依据是《系统需求规格说明书》文档。

（4）验收测试（acceptance testing），指的是在系统测试的后期，以用户测试为主，或有测试人员等质量保障人员共同参与的测试,它也是软件正式交给用户使用的最后一道工序。

其他还有回归测试、冒烟测试、随机测试

其中黑盒测试包括功能测试和性能测试；

功能测试有：逻辑功能测试、界面测试、易用性测试、安装测试、兼容测试；

性能测试有：一般性能测试、稳定性测试、压力测试、负载测试

53.解释体系结构中深度、宽度、扇出、扇入对软件的影响。

体系结构中的深度、宽度、扇出、扇入是指**结构图**的一些特征，它们对软件的影响如下：

- **深度：**指结构图控制的层次，即模块的层数。深度越大，说明模块之间的调用关系越复杂，软件的可维护性和可测试性越低。
- **宽度：**指一层中最大的模块个数。宽度越大，说明模块之间的并行关系越多，软件的可理解性和可维护性越低。
- **扇出：**指一个模块直接下属模块的个数。扇出越大，说明模块之间的耦合度越高，软件的可复用性和可测试性越低。
- **扇入：**指一个模块直接上属模块的个数。扇入越大，说明模块之间的内聚度越高，软件的可复用性和可维护性越高。
- **设计原则：**低扇出高扇入，扇入大表示模块的重用性高,利用率高.扇出大表示模块的复杂度高.所以要高扇入低扇出。

58.什么叫模块化，为什么要划分模块？

模块化是一种软件设计和开发的方法，它将软件系统划分为若干个相对独立的、功能完整的、可复用的模块，每个模块负责一定的功能，模块之间通过接口进行通信和协作。

模块化的目的主要有以下几个：

- 提高软件的可理解性和可维护性，使软件结构清晰，便于分析、设计、编码、测试和修改。
- 提高软件的可复用性和可扩展性，使软件可以重用已有的模块，或者增加新的模块，以适应新的需求或功能。
- 提高软件的可靠性和安全性，使软件可以隔离和控制模块之间的错误或异常，防止错误或异常的传播和影响。
- 提高软件的开发效率和协作效率，使软件可以分工合作，实现并行开发和集成。

59.比较在组装测试中，自顶向下与自底向上、深度优先与宽度优先的优缺点。

自顶向下

优点:

1）较早地验证主要的控制和判断点。

2）可以首先实现和验证一个完整的软件功能，增强信心；

3）减少了测试驱动程序开发和维护的费用;

4）可以和开发设计工作一起并行执行集成测试，能够灵活的适应目标环境；

5）容易进行故障隔离和错误定位。

缺点：桩模块的开发和维护费用大。

自底向上

优点

1）减少了桩模块的工作量;

2）容易对错误进行定位。

缺点

1）直到最后一个模块加进去之后才能看到整个系统的框架；

2）只有到测试过程的后期才能发现时序问题和资源竞争问题；

3）驱动模块的设计工作量大;

4）不能被及时发现高层模块设计上的错误。

自底向上的组装测试方案是工程实践中最常用的测试方法。相关技术也较为成熟。它的优点很明显:管理方便、测试人员能较好地锁定软件故障所在位置。但它对于某些开发模式不适用，如使用 XP 开发方法，它会要求测试人员在全部软件单元实现之前完成核心软件部件的组装测试。尽管如此,自底向上的组装测试方法仍不失为一个可供参考的组装测试方案。

自顶向下集成的优点在于能尽早地对程序的主要控制和决策机制进行检验,因此较早地发现错误。缺点是在测试较高层模块时，低层处理采用桩模块替代，不能反映真实情况，重要数据不能及时回送到上层模块，因此测试并不充分。

自底向上集成方法不用桩模块，测试用例的设计亦相对简单，但缺点是程序最后一个模块加入时才具有整体形象。它与自顶向综合测试方法优缺点正好相反。

•

63.模块化的三个重要特征是什么，阐明各自的作用。

1.相对独立性，可以对模块单独进行设计、制造、调试、修改和存储，这便于由不同的专业化企业分别进行生产；

2.互换性，模块接口部位的结构、尺寸和参数标准化，容易实现模块间的互换，从而使模块满足更大数量的不同产品的需要；

3.通用性，有利于实现横系列、纵系列产品间的模块的通用，实现跨系列产品间的模块的通用。

93.什么是模块独立性，如何衡量模块独立性？

模块独立性指每个模块只完成系统要求的独立的子功能,并且与其他模块的联系最少且接口简单。

模块独立性是指模块内部各部分及模块间的关系的一种衡量标准，由内聚和耦合来度量。

1.内聚：衡量模块内部各部分间的功能相关性。高内聚表示模块内元素紧密协作完成单一功能。

2.耦合：衡量模块间的依赖关系。低耦合表示模块间联系简单，修改不影响其他模块。

两者结合，内聚越高、耦合越低，模块独立性越强，系统更易维护和扩展。

98.什么是耦合，如何衡量模块的耦合度？

耦合是程序结构中各个模块之间相互关联的度量。
它取决于各个模块之间的接口的复杂程度、调用模块的方式以及哪些信息通过接口

89.简述渐增式测试方法与非渐增式测试方法的优劣。

渐增式测试是先从单个模块开始测试,然后每次将测试后的一个模块添加到系统中来进行下一个测试的方法。

非渐增式测试，是指先独立地测试每一模块，然后将所有这些模块连接到一起运行；一下子把所有模块放在一起，并把庞大的程序作为一个整体来测试，测试者面对的情况十分复杂。

非渐增式与渐增式两种测试的区别

非渐增式测试方法需要编写的测试用例较多，工作量较大；渐增式测试方法开销小。

渐增式测试方法发现模块间接口错误早；而非渐增式测试方法晚。

非渐增式测试方法发现错误，较难诊断；而使用渐增式测试方法，如果发生错误则往往和最近加进来的那个模块有关。

渐增式测试方法测试更彻底

渐增式测试方法需要较多的机器时间

使用非渐增式测试方法，可以并行测试。

107、软件测试的策略有哪些？

（1）单元测试
单元测试（uit testig），是指对软件中的最小可测试单元进行检查和验证。
（2） 集成测试
集成测试（itegratio testig），也叫组装测试或联合测试。在单元测试的基础上，将所有模块按照设计要求（如根据结构图）组装成为子系统或系统，进行集成测试。
（3） 系统测试
系统测试（system testig），是对整个系统的测试，将硬件、软件、操作人员看作一个整体，检验它是否有不符合系统说明书的地方。
（4） 确认测试
确认测试（cofirmatio testig），又称有效性测试，是对通过集成测试的软件在模拟的环境下进行的有效性测试，目的是要表明软件是可以工作的，并且符合”软件需求说明书”中规定的全部功能和性能要求。

109、面向对象的测试和传统测试的区别？

面向对象软件抛弃了传统的开发模式,对每个开发阶段都有不同于传统软件测试的要求和结果,已经不可能用功能细化的观点来检测面向对象分析和设计的结果。
因为:

a) 测试的对象不同:传统软件测试的对象是面向过程的软件,一般用结构化方法构建;面向对象测试的对象是面向对象软件,采用面向对象的概念和原则,用面向对象的方法构建。
b) 测试的基本单位不同:前者是模块;面向对象测试的基本单元是类和对象。
c) 测试的方法和策略不同:传统软件测试采用白盒测试,黑盒测试,路径覆盖等方法;面向对象测试不仅吸纳了传统测试方法,也采用各种类测试等方法,而且集成测试和系统测试的方法和策略也很不相同。

7.软件项目管理的主要方面是什么？

1）软件项目计划与组织
2）软件项目成本管理
3）软件项目进度控制
4）软件质量保证
5）软件配置管理
6）生成软件项目管理文档

12.成本估算方法有哪几种？

成本估算方法 就方法论而言，有两种基本的成本估算方法：自顶向下和自底向上。
这两种方法都要求采用某种方法做出估算,可以利用的估算方法有:
1、工作分解结构方法（WBS）
2、专家判断
3、基于问题的估算
4、基于经验模型的估算

42.什么是软件配置管理，有什么作用？

简单而言就是**管理软件的变化**,它应用于整个软件工程过程，通常由相应的工具、过程和方法学组成。
实施有效的软件配置管理可以解决以下软件开发中的常见问题：
1、开发人员未经授权修改代码或文档;
2、人员流动造成企业的软件核心技术泄密;
3、找不到某个文件的历史版本;
4、无法重现历史版本;
4、无法重新编译某个历史版本，使维护工作十分困难;
5、“合版本”时，开发冻结，造成进度延误;
6、、软件系统复杂,编译速度慢,造成进度延误;
7、因一些特性无法按期完成而影响整个项目的进度或导致整个项目失败;
8、已修复的 Bug 在新版本中出现;
9、配置管理制度难于实施;
10、分处异地的开发团队难于协同,可能会造成重复工作,并导致系统集成困难;

47.基线在配置管理中有什么作用？

基线是软件文档或源码的一个稳定版本，它是进一步开发的基础。基线在配置管理中的作用是为开发软件提供了一个定点和快照，可以方便比较、评审、交接、检查、修复、发布等

52.软件维护的基本内容是什么？（软件维护的类型）

软件维护的基本内容是指根据需求变化或硬件环境的变化对应用程序进行部分或全部的修改。软件维护一般分为以下四种类型：

- **改正性维护：**是指改正在系统开发阶段已发生而系统测试阶段尚未发现的错误。
- **适应性维护：**是指使应用软件适应信息技术变化和管理需求变化而进行的修改。
- **完善性维护：**是指为扩充功能和改善性能而进行的修改，主要是指对已有的软件系统增加一些在系统分析和设计阶段中没有规定的功能与性能特征。
- **预防性维护：**是为了改进应用软件的可靠性和可维护性，为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使应用系统适应各类变化而不被淘汰。

101. 简述一下里程碑的作用？（在项目中设置里程碑有什么好处?）

里程碑一般是项目中**完成阶段性工作的标志**，标志着上一阶段的结束、下一阶段的开始。**里程碑定义了当前阶段完成的标注和下一新阶段启动的条件和前提**。

里程碑的作用：

（1） **明确目标与方向**：里程碑帮助团队明确项目的关键目标和方向，确保所有成员对项目的重要阶段有共同的理解和期待。

（2） **监控进度**：通过定期检查里程碑的达成情况，项目经理可以有效监控项目的整体进度，及时发现并解决潜在问题。

（3） **风险控制**：里程碑作为项目的关键节点，有助于识别和评估项目风险，提前采取措施减少不确定性和负面影响。

（4） **激励团队**：达成里程碑是团队努力的结果，可以增强团队士气，提高成员的工作动力和满意度。

（5） **沟通工具**：里程碑是向项目相关方报告项目状态的有力工具，有助于保持透明度，建立信任。

91.软件技术审查和管理复审(评审)的作用是什么？

1、软件技术审查由一组评审者按照规范的步骤对软件需求、设计、代码或其他技术文档进行仔细地检查，以找出和消除其中的缺陷。

作用：

（1）发现软件在功能、逻辑、实现上的错误；

（2）验证软件符合它的需求规格；

（3）确认软件符合预先定义的开发规范和标准；

（4）保证软件在统一的模式下进行开发；

（5）便于项目管理。

2、管理复审是指系统地评价鉴定全部管理工作的一种控制方法，它侧重于管理决策方面的审核，包括对组织中关于计划、工程技术、生产、营销、人事、会计以及财务方面的审

核。

管理复审的主要任务是，在软件生命周期的每个阶段结束时，对工程项目的成本、实际花费的经费、投资回收的前景、项目的进度等经济因素从管理角度进行审查。从而找出在成本、进度、质量上存在的问题，为后续改进提供问题基础。

简版回答：

软件技术审查和管理审查是软件质量保证的两种重要活动。 软件技术审查是对软件开发过程中的各种文档、设计、代码等进行系统性、结构化的检 查，以发现和消除潜在的缺陷，提高软件质量和可靠性。

管理审查是对软件项目的总体状况、成本、进度、风险等进行评估和监控，以便及时调整项目计划，保证项目目标的实现。

105、为什么没有一个单一的、全包容的程序复杂度度量方法?

程序复杂度难以通过单一度量方法全面评估，主要原因如下：

（1）多维复杂性

程序复杂度涉及多个维度，如结构复杂度（控制流、数据流）、认知复杂度（代码可读性）、维护复杂度（扩展性）等。不同维度需不同指标衡量，例如：

圈复杂度（Cyclomatic Complexity）侧重控制流分支；

Halstead 复杂度关注操作符与操作数数量；**代码行数（LOC）**仅反映规模而非逻辑难度。

(2)应用场景的多样性

不同领域对复杂度的定义不同：

嵌入式系统关注实时性和资源占用；

分布式系统聚焦通信与同步逻辑；

业务系统重视可维护性与需求变更适应性。

单一指标无法满足所有场景的核心需求。

(3)语言与范式的差异

编程语言特性（如函数式、面向对象）和架构风格（微服务、单体）直接影响复杂度表现。例如：

函数式编程的递归和高阶函数复杂度难以用传统面向对象指标衡量；

微服务架构的分布式事务复杂度远超单体应用。

(4)主观性与上下文依赖

复杂度感知受开发者经验、团队规范及业务目标影响：

性能优化代码可能牺牲可读性，但对特定场景至关重要；

设计模式可能增加结构复杂度，却提升扩展性。

统一标准无法平衡此类权衡。

(5)度量工具的局限性

现有工具（如静态分析器）通常聚焦技术指标，但忽略：

团队协作成本（如代码评审难度）；

业务逻辑的抽象合理性（如领域模型清晰度）。

总结

程序复杂度本质上是多维度、多视角的综合问题，需结合具体场景、语言特性和团队目标，通过多种指标互补（如圈复杂度+认知复杂度+架构耦合度）进行合理评估，而非依赖单一“万能”方法。

软件复杂性度量是一个复杂的问题，因为软件复杂性是一个多维度的概念，包括代码行数、

代码结构、代码质量、代码可读性、代码可维护性等多个方面。不同的软件项目可能会有不同的复杂性特征，因此很难单一对软件复杂性进行度量。

106、软件开发成本由哪些组成成分？

(1)人员成本：包括开发人员、测试人员、项目经理等人员的工资、福利、培训等费用。
(2)硬件成本：包括服务器、存储设备、网络设备等硬件设备的购置、维护、更新等费用。
(3)软件成本：包括开发工具、测试工具、数据库软件等软件的购置、维护、更新等费用。
(4)外包成本：如果公司选择外包开发，那么外包公司的服务费用也是软件开发成本的一部分。
(5)其他成本：包括办公场地租金、水电费、差旅费等其他费用。

67.简述软件的维护过程。

软件维护的过程一般包括以下几个步骤：

- 接收和分析维护请求，维护人员收到用户或管理者提出的维护请求，对请求进行分析，确定维护的类型、范围、优先级、可行性等。
- 制定和评估维护方案，维护人员根据维护请求，制定一个或多个维护方案，包括维护的目标、方法、工具、资源、时间、成本等，并对方案进行评估和选择。
- 执行和测试维护方案，维护人员按照选定的维护方案，对软件进行修改和更新，并对修改后的软件进行测试和验证，确保软件的正确性和完整性。
- 交付和评价维护结果，维护人员将修改后的软件交付给用户或管理者，并收集用户或管理者对维护结果的反馈和评价，以及对维护过程的改进意见。

62.软件维护有哪些特点？

软件维护是指在软件交付后，对软件进行修改和更新，以适应用户需求的变化，修复软件缺陷，提高软件性能和质量等。
软件维护有以下几个特点：

- 软件维护是一个持续的过程，不是一个一次性的事件，软件维护的周期通常比软件开发的周期长。
- 软件维护是一个复杂的过程，涉及到多方面的因素，如用户、开发者、管理者、环境等，需要进行有效的沟通和协调。
- 软件维护是一个成本高昂的过程，据统计，软件维护的成本占软件生命周期成本的60%~80%。

软件维护是一个风险较大的过程，每次修改都可能引入新的错误或影响原有的功能，需要进行充分的测试和验证。

57.软件维护分几类，每类的内容是什么？

软件维护是指在软件交付后，对软件进行修改和更新，以适应用户需求的变化，修复软件缺陷，提高软件性能和质量等。

软件维护一般分为以下几类：

- **更正性维护**：更正性维护是指对软件中存在的错误或缺陷进行修复，使软件符合规格说明。
- **适应性维护**：适应性维护是指对软件进行修改，以适应外部环境的变化，如硬件、操作系统、协议等。
- **完善性维护**：完善性维护是指对软件进行修改，以增加或改进软件的功能、性能、可用性等，满足用户的新需求或期望。
- **预防性维护**：预防性维护是指对软件进行修改，以预防未来可能出现的问题，提高软件的可维护性和可扩展性。

72.软件的可维护性与哪些因素有关，如何提高软件的可维护性？

主要有 5 个：1 可理解性 2 可测试性 3 可修改性 4 可移植性 5 可重用性

方法：

1. 建立明确的软件质量目标。
2. 采用现代化的开发方法。
3. 建立明确的质量保证审查。
4. 选择可维护的语言。
5. 改进程序员的文档。

7.什么是软件维护的副作用，如何防止软件维护的副作用？

软件维护的副作用主要是在修改原有错误时会引入新的错误。

预防的措施主要是程序模块化，信息隐蔽，高内聚、低耦合等。

面向对象分析设计中使用封装、抽象、继承等办法。具体的包括：

1、按模块把修改分组；

2、自顶向下地分析所修改模块的顺序；自底向上修改有关模块。

3、每次修改一个模块；牵涉到全局性的基础模块改动必须仔细分析。

4、对于每个修改了的模块，在安排修改下一个模块之前，要确定这个修改的副作用，主要是关注修改后对调用这个模块的其他模块的功能影响。

92.软件的可维护性与那些软件质量特性有关？

软件可维护性的定义：软件能够被理解、校正、适应及增强功能的容易程度。

软件可维护性可用下面七个质量特性来衡量，即可理解性、可测试性、可修改性、可靠性、可移植性、可使用性和效率。对于不同类型的维护,这七种特性的侧重点也是不相同。

1.简述软件安全性与软件可靠性的异同。

相同点：软件安全性与软件可靠性均与软件失效有关.

不同点：软件可靠性使用统计分析的方法来确定软件失效发生的可能性,而失效的发生未必导致灾难或灾祸.软件安全则考察失效会导致灾难发生的条件.

6.什么是 McCabe 复杂性度量，如何计算？

是利用程序模块的程序图中环路的个数，来计算程序的复杂性的。为此，也称为环路复杂度计算法。

计算方法：

V(G)=m-n+p

V(G)是有向图 G 中环路数，m 是图 G 中弧数，n 是图 G 中结点数，p **是图 G 中强连通分量个数。**

9.如何估算程序中的错误？

可用下面的估算方法来计算:

1)用 Shooman 模型估算 ET

2)用植入法估算固有错误总数

3)用**最小二乘法估算**固有错误

11.软件的缺陷为什么在软件开发和维护过程中会扩大？

软件的缺陷是指软件产品与需求或规格不一致的地方，它会影响软件的功能、性能、可靠性等。软件缺陷在软件开发和维护过程中会扩大的原因有以下几点：

需求不清晰或频繁变更。如果需求没有明确地定义或者在开发过程中不断地修改，会导致软件产品与需求之间的差距越来越大，从而产生更多的缺陷。

设计不合理或编码错误。如果软件的设计没有考虑到各种情况或者与需求不匹配，或者编码过程中出现逻辑错误或语法错误，会导致软件产品无法正常运行或出现异常，从而产生更多的缺陷。

测试不充分或环境差异。如果软件的测试没有覆盖到所有的功能、场景或边界条件，或者测试环境与实际运行环境有差异，会导致软件产品在实际使用中出现问题，从而产生更多的缺陷。

16.什么是 CMM 关键过程域，举例说明其含义。

关键过程域是指一系列相互关联的操作活动,这些活动反映了一个软件组织改进过程时必须集中精力改进的几个方面。换句话说，关键过程域标识了达到某个成熟度等级时所必须满足的条件。

如软件成熟度模型中第 4 级（管理级）有 2 个关键过程域，**主要的任务是为软件过程和软件产品建立一种可以理解的定量的方式。**

17.影响软件成本的主要因素有哪些具体方面？

软件的规模、复杂度、开发工具、开发人员的业务素质、软件工程管理水平及开发工作环境等。

21.软件质量的特性有哪些，怎样度量？

功能性、可靠性、易用性、效率、可维护性、可移植性。

通**过产品质量、过程中质量和软件维护质量**度量等三种方法来度量。

22.什么是 COCOMO 成本估算模型？

是一种利用经验模型进行成本估算的方法

COCOMO，英文全称为 constructive cost model，中文为构造性成本模型。它是一种精确、易于使用的，基于模型的成本估算方法，最早由勃姆 （Boehm） 于 1981 年提出。从本质上说是一种参数化的项目估算方法，参数建模是把项目的某些特征作为参数，通过建立一个数字模型预测项目成本（类似于居住面积作为参数计算的整体的住房成本）。

26.Boehm 和 McCall 等人从哪些方面来确定软件质量，他们有什么异同？

软件质量就是“软件与明确的和隐含的定义的需求相一致的程度”。具体地说，软件质量是软件符合明确叙述的功能和性能需求、文档中明确描述的开发标准、以及所有专业开发的软件都应具有的隐含特征的程度。 影响软件质量的主要因素，这些因素是从管理角度对软件质量的度量。可划分为三组，分别反应用户在使用软件产品时的三种观点。正确性、健壮性、效率、完整性、可用性、风险（产品运行）；可理解性、可维修性、灵活性、可测试性（产品修改）；可移植性、可再用性、互运行性（产品转移）。

McCall 等人将质量模型分为三层因素、衡量准则、度量,并对软件质量因素进行了研究,认为软件质量是正确性、可靠性、效率等构成的函数,而正确性、可靠性、效率等被称为软件质量因素,或软件质量特征,它表现了系统可见的行为化特征。每一因素又由一些准则来衡量,而准则是跟软件产品和设计相关的质量特征的属性。

Boehm 模型是由 Boehm 等在 1978 年提出来的质量模型,在表达质量特征的层次性上它与 McCall 模型是非常类似的。不过,它是基于更为广泛的一系列质量特征,它将这些特征最终合并成 19 个标准。Boehm 提出的概念的成功之处在于它**包含了硬件性能的特征**,这在 McCall 模型中是没有的。但是,其中与 McCall 模型类似的问题依然存在。

27.什么是软件的逆向工程和再工程？

软件逆向工程是通过反汇编和调试等手段分析计算机程序的二进制可执行代码,从而获得程序的算法细节和实现原理的技术

软件再工程是对既存软件系统进行调查，**并将其重构为新形式代码的开发过程**。

29.纠错技术中的归纳法和演绎法的中心思想是什么？

1.归纳法，指的是从许多个别事例中获得一个较具概括性的规则。这种方法主要是从收集到的既有资料，加以抽丝剥茧地分析，最后得以做出一个概括性的结论。演绎法，则与归纳法相反，是从既有的普遍性结论或一般性事理，推导出个别性结论的一种方法。由较大范围，逐步缩小到所需的特定范围。

2.归纳法是从特殊到一般，优点是能体现众多事物的根本规律，且能体现事物的共性。缺点是容易犯不完全归纳的毛病。演绎法是从一般到特殊，优点是由定义根本规律等出发一步步递推，逻辑严密结论可靠，且能体现事物的特性。缺点是缩小了范围，使根本规律的作用得不到充分的展现。归纳法和演绎法在应用上并不矛盾，有些问题可采用前者，有些则采用后者。而更多情况，将两者结合着应用，则能收到更好的效果。

**31.影响软件质量的因素分哪两大类？（或影响软件质量的因素有哪些？）*

（1）可直接度量的因素：如单时间内千行代码中所产生的错误数
（2）不可直接度量的因素：如可用性可维护性等
软件质量因素集中在产生的三个重要方面：
- 软件的操作特性
- 软件承受变化的能力
- 对新环境的适应性

32.如何衡量软件成本，各表示什么含义？ *

成本估算和成本管理是软件项目管理的核心任务之一。在制定项目计划时，就必须对项目需要的人力及其他资源、项目持续时间和项目成本做出估算。如果新项目和以往的项目类似，估算可以参考以前的成本费用。

软件工程中，软件成本的衡量方法主要有以下几种：

- 功能点法：根据软件的功能复杂度和规模，计算软件的功能点数，再乘以每个功能点的成本，得到软件的总成本。
- COCOMO 法：根据软件的类型、规模、开发难度等因素，使用数学模型估算软件的开发时间和人力，再乘以每人每月的成本，得到软件的总成本。
- 专家判断法：根据专家的经验和知识，对软件的开发过程和资源进行评估，给出软件的总成本。
- 类比法：根据与当前项目相似的历史项目的数据，对软件的开发过程和资源进行比较和调整，得到软件的总成本。

36.软件质量的含义是什么？

软件质量是软件特性的综合，指软件满足规定或潜在用户需求的能力。

- **内部质量**：指软件产品本身的质量，如代码的可读性、可维护性、可测试性等，它可以通过静态分析和代码审查等方法来评估。
- **外部质量**：指软件产品在运行时的质量，如功能性、可靠性、效率、可用性等，它可以通过动态测试和用户反馈等方法来评估。
- **使用质量**：指软件产品在特定环境和条件下的质量，如满足性、安全性、适应性等，它可以通过用户调查和市场分析等方法来评估。
- **过程质量**：指软件开发过程中遵循的标准和规范的质量，如计划、管理、监控、改进等，它可以通过过程评估和过程改进等方法来评估。

45.领域分析的目标是什么，依据是什么？

领域分析的目标是理解和描述一个特定的问题域，包括它的概念、术语、规则、实体、关系、行为、过程等。领域分析的依据是问题域的需求和特征，以及相关的文献、资料、专家、用户等。领域分析可以帮助提高软件开发的效率和质量，以及促进软件复用和维护。

97.CMMI 与 ISO9000 有什么异同

1、CMMI 是指“能力成熟度模型集成”，其英文全称为 Capability Maturity Model Intergration，简称 CMMI。它是对于软件组织在定义、实施、度量、控制和改善其软件过程的实践中各个发展阶段的描述。CMMI 的核心是把软件开发视为一个过程，并根据这一原则对软件开发和维护进行过程监控和研究，以使其更加科学化、标准化、使企业能够更好地实现商业目标。

2、ISO9000 是指质量管理体系标准，它不是指一个标准，而是一种标准的统称。

ISO 与 CMM 的相同：

(1)ISO9000 质量体系与 CMMI，都共同着眼于质量和过程管理，两者都为了解决同样的问题。从一方面说他们是相互联系、相互补充的。

(2)两者都吸收了现代质量管理理论，都以“过程思维”为指导。

(3)ISO9001 中的质量要素都可以对应到 CMMI 中关键过程区域特征上，而 CMMI 在生产过程中的管理重点，又弥补了 ISO9001 在微观管理上的不足。

ISO 与 CMM 的不同：

（1）ISO9001 确定一个质量体系的最少需求，而 CMMI 模型更在注重持续过程改进。

（2）而且，ISO9001 只建立了一个可接受水平，而 CMMI 是一个具有五个水平的评估工具。

（3）ISO9001 标准面向合同环境，站在用户立场，要求对质量要素进行控制，规定了质量体系的最低标准。而 CMMI 标准则强调软件开发过程的成熟度，即内部过程的不断改进和提高。

（4）在形式上，ISO9001 审核只有“通过”和“不通过”两个结论，而 CMMI 评定则是一个动态过程，软件组织在通过低级别评估后，可根据高级别的要求确定改进方略。

15.针对软件危机，论述传统软件工程方法和敏捷方法的解决思路

PPT 参考：

软件危机出现于 20 世纪 60 年代末，软件危机是指在计算机软件的开发和维护过程中遇到的一系列严重问题。比如：

1）软件开发的成本和进度难以准确估计，延迟交付甚至取消项目的现象屡见不鲜

2）软件存在着错误多、性能低、不可靠、不安全等质量问题

3）软件维护极其困难，而且很难适应不断变化的用户需求

为了解决“软件危机”，许多计算机和软件科学家参照技术过程的一般模式提出了软件生存期概念（Life cycle）及其瀑布模型（Waterfall Model）。计算机软件的生存周期描述了软件孕育、诞生、成长、成熟、衰亡的生存过程：

制定计划、需求分析、软件设计、程序编制、软件测试、运行维护。

但是，传统软件工程方法推行多年以来，人们对于软件危机的“恐惧”仍没有丝毫减弱，相

反随着软件系统的急速膨胀而增强，表现为：对软件开发成本和进度的估计常常不准确，开发成本超出预算，实际进度比预定计划一再拖延的现象并不罕见；用户对“已完成”系统不满意的现象也经常发生；软件产品的质量往往靠不住，Bug 一大堆，补丁一个接一个等等。于是，无论是产业界还是理论界，都开始对传统软件工程学思想产生怀疑，甚至背叛，于是以“人”为核心的敏捷开发方法就是在这样的背景下产生的，它背叛了传统软件工程学中以“过程”为核心，把设计和开发尽可能分开，尽量弱化“人”在整个工程中地位的思想。敏捷开发实际上包括了许多优秀的软件开发习惯。首先，这种方法改变了软件测试的流程，在编写代码前进行测试，减少了开发风险；此外，通过这种方法，可以对软件进行持续集成，即每个小时都在集成，任何部件间的冲突都可以随时解决；这种方法的“动态规划”和“重构”做法，意味着开发者可以对软件架构进行持续改进，可以根据用户的需求随时进行改进，而利用传统的软件开发方法则很难对软件的架构进行调整，同时也会造成成本的大幅增加。

5.论述需求工程过程，说明各阶段关系

答：需求工程过程包括需求开发和需求管理两个部分，需求开发的一般过程分为需求获取、需求分析、编写需求规格说明书（SRS）、需求验证四个阶段；

需求管理则主要包括需求基线的建立、需求变更控制以及需求跟踪等活动。需求获取是通过与用户的交流，对现有系统的观察及对任务进行分析，从而开发、捕获和修订用户的需求；需求分析是通过之前需求获取所得到的信息，为最终用户所看到的系统建立一个概念模型，作为对需求的抽象描述；由需求模型构件生成精确的形式化的描述，即需求规格说明书，它将作为用户和开发者之间的一致协议；

需求分析的结果应该通过评审、测试等手段验证它的正确性、完整性和一致性，这就是需求验证。

贯穿于整个过程中，需求管理中最基本的任务则是明确需求，并使所有相关人员达成共识；建立需求跟踪能力联系链，确保所有用户需求被正确地应用，并且在需求发生变更时，能够完全地控制其影响范围，始终保持产品与需求的一致性。

7.论述软件工程中的文档类型和作用. *

答：**按照文档产生和使用的范围，软件文档大致可分为三类：开发文档、管理文档、用户文档。**

一、开发文档

定义：**开发文档是描述软件技术实现细节的文档，供开发、测试和维护人员使用，贯穿需求、设计、编码、测试等阶段。**

1. 需求规格说明书（SRS）

作用：

明确系统功能、性能、接口等需求，作为设计与验收的基准。

通过用例、数据流图等形式描述用户需求，避免开发偏差。

示例内容：用户场景、非功能性需求（如并发量、响应时间）、数据字典。

2. 系统设计文档（SDD）

作用：

定义系统架构（如分层架构、微服务）、模块划分及交互逻辑。

指导开发团队分工协作，确保技术方案一致性。

示例内容：架构图、模块接口定义、数据库 ER 图、API 规范。

3. 详细设计说明书

作用：

细化模块内部逻辑，指导具体编码实现。

提供算法描述、类图、流程图等，降低编码复杂度。

示例内容：类属性与方法定义、伪代码、输入输出规则。

4. 测试计划与用例

作用：

规划测试策略（单元测试、集成测试、性能测试等），确保需求覆盖率。

提供可执行的测试场景，验证系统是否符合预期。

示例内容：测试环境配置、测试用例（正向/反向场景）、缺陷追踪流程。

5. 技术白皮书

作用：

解释核心技术的实现原理（如加密算法、分布式架构），供技术评审或外部合作参考。

示例内容：技术选型依据、性能优化方案、第三方库依赖说明。

二、管理文档

定义：管理文档用于项目规划、资源协调和过程控制，服务于项目经理、质量保证团队及利益相关者。

1. 项目计划书

作用：

明确项目目标、里程碑、资源分配（人力、预算、时间）。

作为项目执行的基准，指导团队分工与进度跟踪。

示例内容：WBS（工作分解结构）、甘特图、风险管理计划。

2. 进度报告

作用：

定期同步项目进展，暴露延期风险或资源瓶颈。

向管理层汇报关键成果和下一步计划。

示例内容：已完成任务、未完成任务及原因、调整后的时间表。

3. 变更控制文档

作用：

记录需求或设计的变更请求，评估影响并跟踪执行。

防止无序变更导致项目失控。

示例内容：变更申请单、影响分析（成本、进度）、变更审批记录。

4. 质量保证计划（QAP）

作用：

制定质量标准（如代码覆盖率、缺陷密度），确保交付质量。

定义代码审查、测试覆盖等质量控制流程。

示例内容：质量指标、审计计划、验收标准。

5. 风险管理计划

作用：

识别潜在风险（如技术难点、第三方依赖延迟），制定应对策略。

降低项目失败概率。

示例内容：风险清单（概率/影响矩阵）、应急预案、风险责任人。

三、用户文档

定义：用户文档面向最终用户或运维人员，帮助其理解、使用和维护系统。

1. 用户手册

作用：

提供系统操作指南（如功能使用步骤、界面说明），降低用户学习成本。

常见形式：PDF 手册、在线帮助页面。

示例内容：登录流程、功能截图、常见问题解答（FAQ）。

2. 安装部署指南

作用：

指导运维人员完成系统安装、环境配置（如服务器、数据库）。

确保系统正确部署并运行。

示例内容：软硬件要求、配置参数说明、故障排查步骤。

3. 维护手册

作用：

描述系统维护流程（如数据备份、日志清理）、故障恢复方法。

延长系统生命周期，保障稳定性。

示例内容：日常维护任务清单、错误代码解释、联系支持渠道。

4. 发布说明（Release Notes）

作用：

通告版本更新内容（新增功能、修复缺陷），帮助用户了解升级影响。

提供升级步骤和兼容性说明。

示例内容：版本号、更新日志、已知问题、升级注意事项。

5. 培训材料

作用：

辅助用户或运维人员快速掌握系统核心功能。

常见形式：PPT 课件、操作视频、模拟演练脚本。

示例内容：培训大纲、实操案例、考核试题。

四、文档的核心价值

知识传递：沉淀技术方案与管理经验，减少人员流动带来的信息丢失。

过程可控：通过规范化文档确保开发流程透明、可追溯。

质量保障：明确需求与验收标准，降低交付偏差风险。

协作基础：为跨团队、跨角色沟通提供统一依据。

通过系统化的文档管理，软件工程活动得以从“人治”转向“法治”，显著提升项目的可预测性和成功率。

8.论述需求工程的过程和方法，讨论其有效性

答：需求工程包括需求开发和管理，而需求开发又包括这几个过程：需求获取，需求分析，需求规格说明和需求验证。

在需求开发之前，还需要有一个知识培训的过程，需求工程也是一个项目工程，因此也包括了项目的管理。

对于这些过程，有以下方法可以采用。

1、知识培训：需求分析员培训、用户培训、开发人员培训；

2、需求获取：项目范围确定、用户确定、用例确定、系统事件和响应、获取方法、检查完善；

3、需求分析：绘制关联图、原型开发、需求优先顺序别、需求建模、数据字典创建、子系统。

在知识培训中，通过对三方的培训，可以使分析员提高沟通能力，理解并掌握需求工程技能，使用户明白需求重要性，能够准确描述需求，使开发人员对用户应用有所了解。

通过需求获取中的方法，可以明确项目目标、确定用户群，通过用例迅速明确项目功能需求，这些可以通过会议讨论，观察用户工作过程，问答式对话，诱发式需求诱导等实现。

而在需求分析阶段则是对用户的需求获取之后的一个粗加工过程，需要对需求进行推敲和润色以使所有涉众都能准确理解需求。 分析过程首先需要对需求进行检查，以保证需求的正确性和完备性，然后将高层需求分解成具体的细节，创建开发原型，完成需求从需求获取人员到开发人员的过渡。

25、论述软件过程改进对软件工程的意义和作用。

在当今信息技术快速发展的时代，软件已经成为各个行业和领域的重要组成部分。为了保证软件的质量和可靠性，软件工程过程的改进变得尤为重要。本文将从软件工程的定义、软件过程改进的意义、软件过程改进的方法和实施过程等方面进行分析和探讨。

一、软件工程的定义

软件工程被定义为一种通过科学和工程化的方法，以规范化的过程、原则和方法来开发和维护软件系统的学科。软件工程的目标是提高软件的开发效率、质量和可维护性，并确保软件满足用户需求。

二、软件过程

软件过程是指 在软件工具的支持下,所进行的一系列软件开发和进化的活动。

三、软件过程改进的意义

1. 提高软件质量：软件过程改进可以通过引入有效的软件质量管理方法和工具，提高软件开发过程中的质量控制和检测，从而提高最终软件产品的质量。

2. 提高软件开发效率：通过软件过程改进，可以规范和优化软件开发过程中的各个环节，提高开发效率，减少开发周期，降低开发成本。

3. 降低风险：软件过程改进可以帮助识别和管理软件开发过程中的风险，减少项目失败的概率，保证软件项目的成功交付。

四、软件过程改进的方法

1. CMMI（Software Capability Maturity Model Integration）模型：CMMI 是一种用于评估和改进软件开发过程的模型。它提供了一套评估标准和最佳实践，帮助组织识别并改进软件开发过程中的问题，提高组织的软件工程能力。

2. 敏捷开发方法：敏捷开发是一种迭代、增量、以人为中心的软件开发方法。它强调快速反馈、灵活性和高度合作的团队工作方式，通过多次迭代开发，逐步完善软件系统。

3. 六西格玛方法：六西格玛方法是一种通过测量和分析数据，减少过程中的变异性和缺陷数量的方法。它强调使用统计方法和数据驱动的决策，从而提高软件过程的稳定性和可预测性。

五、软件过程改进的实施过程

1. 确定改进目标：首先，需要明确软件过程改进的目标和期望效果，明确改进的方向和重点。
2. 进行现状评估：通过对当前软件开发过程的评估，分析现状，识别存在的问题和瓶颈，并确定改进的优先级和措施。
3. 设计改进计划：根据评估结果，制定详细的软件过程改进计划，包括改进的方法、步骤、时间计划和资源安排等。
4. 实施改进措施：根据改进计划，逐步实施改进措施，引入新的方法、工具和技术，优化软件开发过程中的各个环节。
5. 监控和评估：在改进过程中，需要不断监控和评估改进效果，及时调整和纠正，确保软件过程改进取得可持续的效果。
总结：
软件过程改进是提高软件开发过程质量和效率的重要手段，通过引入适当的改进方法和工具，可以帮助组织规范和优化软件开发过程，提高软件质量和开发效率，并降低风险。在实施软件过程改进时，需明确目标、评估现状、设计改进计划、实施改进措施，并持续监控和评估。只有不断改进和优化软件过程，才能适应快速变化的市场需求，保持竞争优势。

21、论述软件开发中的不确定性及应对方法?

软件开发中的不确定性来源
1.需求不确定性：客户需求模糊、频繁变更或隐性需求未被识别。
案例：某电商项目初期仅要求“支持在线支付”，开发中客户突然要求集成加密货币支付，导致架构需重构。
2. 技术不确定性：新技术选型风险（如性能不足）、第三方服务不可靠（如 API 接口变动）。
案例：团队采用某开源框架开发，版本升级后出现兼容性问题，修复耗时两周。
3. 过程不确定性：任务估算偏差、沟通低效、团队协作问题。
案例：瀑布模型中，测试阶段发现需求理解错误，返工成本高昂。
4. 环境不确定性：政策法规变化（如数据隐私法）、市场竞争压力或突发危机（如疫情导致远程协作）。
案例：某医疗软件因 GDPR 法规调整，需紧急重构数据存储逻辑。
研发项目中的不确定性和复杂性是常见的挑战，需要采取一系列的策略和方法来应对。下面从需求管理、团队合作、资源规划、风险管理和沟通协调等方面分别进行讨论。
一、需求管理
1.清晰的需求定义：确保项目团队对于项目的目标和要求有清晰一致的理解，减少后期变更和不确定性带来的影响。
2.迭代开发：采用敏捷开发方法，将项目拆分成多个迭代周期，每个周期内集中处理一部分需求，以降低整体不确定性和复杂性的影响。
3.需求优先级排序：根据项目的关键性和风险程度，对需求进行优先级排序，先解决高优先级的需求，减少不确定性对项目进度的影响。
二、团队合作
1.构建高效团队：选择合适的团队成员，具备专业知识和沟通能力，能够快速适应变化和解决问题。
2.跨部门协作：加强与其他团队和部门的协作，及时沟通和共享信息，减少信息孤岛和决策

不一致的风险。
3.良好的沟通与协调：建立良好的沟通机制，保持团队成员之间的密切合作和信息的畅通流动，及时解决问题。
三、资源规划
1.合理的资源调配：根据项目的不确定性和复杂性的变化，灵活调整人力、物力和财力的分配，保证项目顺利进行。
2.资源约束管理：在资源有限的情况下，合理优化资源利用效率，减少资源的浪费和冲突，提高项目的成功率。
3.预留资源缓冲：考虑到可能出现的不确定性和复杂性，适当预留一部分资源作为缓冲，以应对项目过程中的意外情况。
四、风险管理
1.风险识别与评估：及早识别和评估项目中的风险，制定相应的应对措施，降低不确定性和复杂性的影响。
2.风险监控与控制：定期进行风险管理的检查和评估，及时调整项目计划和资源配置，控制风险的扩大。
3.持续学习与改进：根据项目过程中的不确定性和复杂性的经验教训，总结经验，提炼方法论，为未来的项目提供参考和改进。
五、沟通协调
1.定期项目会议：定期召开项目会议，分享项目进展，解决问题，及时调整项目方向和策略，保持团队的信息共享和协作。
2.有效沟通渠道：建立多样化的沟通渠道，包括面对面、在线会议、邮件等，保持及时和有效的沟通和信息流通。
3.情绪管理和冲突解决:及时处理团队成员之间的冲突和压力,保持团队合作和项目稳定性。
总结而言，研发项目中的不确定性和复杂性需要通过需求管理、团队合作、资源规划、风险管理和沟通协调等方法来应对。在实践过程中，灵活应用合适的策略和方法，能够更好地应对项目中的变化和挑战，提高项目的成功率和交付质量。

13.讨论不同软件过程模型与软件开发的关系

常见的过程模型有瀑布模型、快速原型模型、增量模型、螺旋模型、喷泉模型等。

软件过程是为了获得高质量软件所需要完成的一系列任务的框架,它规定了完成各项任务的工作步骤。通常使用生命周期模型简洁地描述软件过程。生命周期模型规定了把生命周期划分成哪些阶段及各个阶段的执行顺序，因此，也称为过程模型。常见的过程模型有瀑布模型、快速原型模型、增量模型、螺旋模型、喷泉模型等。

1.瀑布模型

这个特点有两重含义：

1.必须等前一阶段的工作完成之后，才能开始后一阶段的工作；

2.前一阶段的输出文档就是后一阶段的输入文档,因此,只有前一阶段的输出文档正确,后一阶段的工作才能获得正确的结果。

瀑布模型每个阶段都应坚持两个重要做法：

1.每个阶段都必须完成规定的文档，没有交出合格的文档就是没有完成该阶段的任务。完整、准确的合格文档是软件开发时期各类人员之间相互通信的媒介，也是运行时期对软件

进行维护的重要依据。

2.每个阶段结束前都要对所完成的文档进行评审，以便迟早发现问题，改正错误。事实上越是早期阶段犯下的错误，暴露出来的时间就越晚，排除故障改正错误所需付出的代价也越高。因此，及时审查，是保证软件质量，降低软件成本的重要措施。

可以说瀑布模型是由文档驱动的。这个事实也是它的一个缺点，在可运行的软件产品交付给用户之前，用户只能通过文档来了解产品是什么样的。瀑布模型历史悠久、广为人知的，它的优势在于它是规范的、文档驱动的方法；这种模型的问题是，最终开发出的产品可能并不是用户真正需要的。

（1）传统的瀑布模型：（2）实际的瀑布模型：

2.快速原型模型

所谓快速原型是快速建立起来的可以在计算机上运行的程序，它所能完成的功能往往是最终产品能完成的功能的一个子集。快速原型的本质是“快速”，开发人员应该尽可能快地建造出原型系统，以加速软件开发过程，节约软件开发成本。原型的用作是获知用户的真正需求，一旦需求确定了，原型系统将被抛弃。

快速原型模型正是为了克服瀑布模型的缺点而提出来的。它通过快速构建一个可在计算机上运行的原型系统，让用户试用原型系统并收集用户反馈意见的办法，获取用户的真实需求。

3.增量模型

增量模型也称为渐增模型，使用增量模型开发软件时，把软件产品作为一系列的增量构件来设计、编码、集成和测试。每个构件由多个相互作用的模块构成，并且能够完成特定的功能。使用增量模型时，第一个增量构件往往实现软件的基本需求，提供最核心的功能。

优点：

1.能在较短的时间内向用户提交可完成部分工作的产品。

2.逐步增加产品功能可以使用户有充裕的时间学习和适应新产品，从而减少一个全新的软件可能给客户组织带来的冲击。

增量模型具有可在软件开发的早期阶段使投资获得明显回报和较易维护的优点，但是，要求软件具有开放的结构是使用这种模型时固有的困难。

4.螺旋模型

螺旋模型的基本思想就是，使用原型及其他方法来尽量降低风险。理解这种模型的一个简便方法，是把它看作每个阶段之前都增加了风险分析过程的快速原型模型。

螺旋模型主要适用于内部开发的大规模软件项目。如果进行风险分析的费用接近整个项目的经费预算，则风险分析是不可行的。事实上项目越大，风险也越大，因此进行风险分析的必要性也越大。此外只有内部开发的项目，才能在风险过大时方便中止项目。

螺旋模型的主要优势在于，它是风险驱动，但是，这也可能是它的一个弱点。除非软件开发人员具有丰富的风险评估经验和这方面的专门知识，否则将出现真正的风险：当项目实际上正在走向灾难时，开发人员可能还认为一切正常。

风险驱动的螺旋模型适用于内部开发的大型软件项目，但是，只有在开发人员具有风险分析和排除风险的经验及专门知识时，使用这种模型才会获得成功。

（1）简化的螺旋模型 （2）完整的螺旋模型

4.喷泉模型

喷泉模型对软件复用和生存周期中多项开发活动的集成提供了支持，以面向对象的软件开发方法为基础，它适合面向对象的开发方法。它克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。喷泉模型使开发过程具有迭代性和无间隙性。系统某个部分常常重复工作多次，相关功能在每次迭代中随之加入演化的系统。无间隙是指在分析、设计和实现等

开发活动之间不存在明显的边界。

按照在软件生命周期过程中应完成的任务的性质，在概念上可以把软件生命周期划分成定义、可行性研究、需求分析、总体设计、详细设计、编码和单元测试、综合测试以及运行维护等 8 个阶段。实际从事软件开发工作时，软件规模、种类、开发环境及使用的技术方法等因素，都影响各阶段的划分。

软件过程是为了获得高质量的软件产品所需要完成的一系列任务的框架，它规定了完成各项任务的工作步骤。由于没有适用所有软件项目的任务集合，科学、有效的软件过程应该定义一组适合所承担的项目特点的任务集合。通常使用软件过程模型简洁地描述软件过程，它规定了把软件生命周期划分成的阶段及各个阶段的顺序。

16. 论述过程、方法和工具在软件工程实践中的关系

软件过程、方法和工具是软件工程的三个核心要素。

其中，软件过程是指在软件开发的 整个生命周期中，按照一定的规范和流程进行的一系列活动，包括需求分析、设计、编 码、测试、维护等。

软件方法是指在软件开发过程中，采用的一种或多种技术和方法 论，如面向对象、结构化分析与设计等。软件方法是为软件开发提供了“如何做”的技 术。

而软件工具则是指在软件开发过程中使用的各种辅助工具。软件过程将各个技术层 次结合在一起，这三者之间相互依存，相互作用，共同构成了软件工程的核心体系，使 得合理及时地开发计算机软件成为可能。

20. 什么是软件性能，论述软件开发中如何提高软件的性能

软件的性能是软件的一种非功能特性，它关注的不是软件是否能够完成特定的功能，而是在完成该功能时展示出来的及时性。由于感受软件性能的主体是人，不同的人对于同样的软件能有不同的主观感受，而且不同的人对于软件性能关心的视角也不同。包括系统响应时间和应用延迟时间、吞吐量、并发用户数、资源利用率。

1.论述面向对象方法对传统方法的优势和问题

答：面向对象方法(Object-Oriented Method)是一种把面向对象的思想应用于软件开发过程中，指导开发活动的系统方法，简称 OO (Object-Oriented)方法，是建立在“对象”概念基础上的方法学。对比传统方法，它的优点是：所开发的程序是面向对象程序，直接描述客观世界的对象及其相互关系；具有模块性、继承性和类比性；可广泛支持对象的重用；十分有利于开发过程的质量控制；极大地减小了系统维护的复杂程度。缺点是：面对大型系统时，若缺乏整体系统设计划分，易造成系统结构不合理、各部分关系失调等问题；只能在现有业务基础上进行分类整理，不能从科学管理角度进行理顺和优化；需要一定的软件支持环境。

2.什么是面向对象开发过程，讨论各阶段任务和要点

答：面向对象开发过程就是指利用面向对象方法进行的软件开发的完整流程，主要分为四个阶段：分析、设计、编程、测试。面向对象分析包括需求分析和需求模型化两个部分。其主要作用是明确用户的需求，并用标准化的面向对象模型来规范地表达这一需求，最后形成面向对象的分析模型。它是系统设计的依据。面向对象设计的任务是把分析阶段得到的问题域需求转变成符合要求的、抽象的系统实现方案，主要分为两个阶段：系统设计和对象设计。要点设计针对问题域子系统、人-机交互子系统、任务管理子系统、数据管理子系统、类中的服务和关联。编程和测试则分别是由开发人员编写程序具体代码并在系统代码开发完成后对系统整体性能和功能进行测试。

4.结合软件工程要素，论述面向对象方法的思想

答：软件工程的方法（开发方法）、工具（支持方法的工具）、过程（管理过程）称为软件工程的三要素。方法支撑过程和工具；过程和工具促进方法学的研究。

面向对象方法（Ohject--Oriented Method，OOM）是一种运用对象、类、消息传递、继承、封装、聚合、多态性等概念来构造软件系统的软件开发方法。

它的优点是：所开发的程序是面向对象程序，直接描述客观世界的对象及其相互关系；具有模块性、继承性和类比性；可广泛支持对象的重用；十分有利于开发过程的质量控制；极大地减小了系统维护的复杂程度。

缺点是：面对大型系统时，若缺乏整体系统设计划分，易造成系统结构不合理、各部分关系失调等问题；只能在现有业务基础上进行分类整理，较难从科学管理角度进行理顺和优化；需要一定的软件支持环境。

因此，通过三要素的关系可以看出，面向对象方法决定了它需要一定的开发工具和严格合理的管理过程来支持，同时，在这两个条件得到满足的情况下，面向对象的方法将能够极大的方便、优化工程的开发以及管理维护。

9.讨论软件测试与软件可靠性的关系，如何计算可靠性

答：软件测试的目的是发现软件错误，排除软件缺陷，提升软件可靠性。软件可靠性可定义为在规定的条件下，在规定的时间内，软件不引起系统失效的概率或在规定的时间周期内，在所述条件下程序执行所要求的功能的能力。

据此，可以在测试时间内，对软件各个模块进行测试统计，得到每个功能块的可靠性，功能串联时可靠性 R=R1*R2*•••RN，并联时 R=1-（1-R1）（1-R2）•••（1-RN）。混联则综合两式进行计算。

10.什么是软件可测试性，讨论软件工程与软件可测试性

答：软件的可测试性是指软件发现故障并隔离、定位其故障的能力特性，以及在一定的时间和成本前提下，进行测试设计、测试执行的能力。软件可测试性就是一个计算机程序能够被测试的容易程度。软件工程是一门研究用工程化方法构建和维护有效的、实用的和高质量的软件的学科。

软件工程的目标是：在给定成本、进度的前提下，开发出具有适用性、有效性、可修改性、可靠性、可理解性、可维护性、可重用性、可移植性、可追踪性、可互操作性和满足用户需求的软件产品。

而软件在完成后，必须保证其能够防止因概念、设计和结构等方面的不完善造成的软件系统失效，具有挽回因操作不当造成软件系统失效的能力，也就是要达成软件可靠性。软件测试正是保证软件可靠性的一项非常重要的技术保障。因此，在软件工程中，必须在软件设计时就考虑好软件的可测试性，这样才能在软件完成后通过测试，调整改进软件的不足，满足其可靠性要求

11.论述软件测试的基本任务，方法和策略及其在现代软件工程的地位

测试的目的（来源老师 PPT）：

1）.测试是为了证明程序有错，而不是证明程序无错误；

2）.一个好的测试用例在于能够发现至今未发现的错误；

3）.一个成功的测试是发现了至今未发现的错误的测试。

测试的方法：

1）测试方法概述

2）白盒测试方法

3）黑盒测试方法

4）其他测试方法

测试的策略：

按阶段进行测试是一种基本的测试策略

1）单元测试

2）集成测试

3）系统测试

4）确认测试

软件测试在软件工程中的地位：覆盖整个生命周期。从需求阶段已介入，编写测试计划、测试用例、测试总结等。

12.论述软件测试与软件质量的关系

软件质量是指软件产品的特性可以满足用户的功能、性能需求的能力。软件过程是人们通常所说的软件生命周期中的活动，一般包括软件需求分析、软件设计、软件编码、软件测试、交付、安装和软件维护。随着软件过程的开始，软件质量也逐渐建立起来。软件过程的优劣决定了软件质量的高低，好的过程是高效高质量的前提。人员和过程是决定软件质量的关键因素。高质量的人员和好的过程应该得到好的产品。

软件系统的开发包括一系列生产活动，其中由人带来的错误因素非常多，错误可能出现在程序的最初需求分析阶段，设计目标可能是错误的或描述不完整，也可能在后期的设计和开发阶段，因为人员之间的交流不够，交流上有误解或者根本不进行交流，所以尽管人们在开发软件的过程中使用了许多保证软件质量的方法和技术，单开发出的软件中还会隐藏许多错误和缺陷。可见，只有通过严格的软件测试，才能很好的提高软件质量，而软件质量并不是依靠软件测试来保证的，软件的质量要靠不断的提高技术水平和改进软件开发过程来保证，软件测试只是一种有效的提高软件质量的技术手段，而不是软件质量的安全网。

软件测试能够找出软件缺陷，确保软件产品满足需求。但是测试不是质量保证，二者并不等同。测试可以查找错误并进行修改，从而提高软件产品的质量。软件测试避免错误以求高质量，并且还有其他方面的措施以保证质量问题,如软件质量保证。

正规的软件测试系统主要包括：制定测试计划、测试设计、实施测试、建立和更新测试文档。而软件质量保证的工作主要为：制定软件质量要求、组织正式度量、软件测试管理、对软件的变更进行控制、对软件质量进行度量、对软件质量情况及时记录和报告。软件质量保证的职能是向管理层提供正确的可行信息，从而促进和辅助设计流程的改进。软件质量保证的职能还包括监督测试流程，这样测试工作就可以被客观地审查和评估，同时也有助于测试流程

的改进。二者的不同之处在于软件质量保证工作侧重对软件开发流程中的各个过程进行管理与控制，杜绝软件缺陷的产生。而测试则是对已产生的软件缺陷进行修复。

3.用覆盖的观点讨论面向对象的软件测试策略

答：软件测试覆盖包括分支覆盖，语句覆盖、条件覆盖等。

语句覆盖：它要求被测程序的每一可执行语句在测试中尽可能都检验过;

分支覆盖：要求程序中所有判定的分支尽可能得到检验;

条件覆盖：当判定式中含有多个条件时，要求每个条件的取值均得到检验;

还有其他的覆盖方式则是将这三种覆盖进行组合，使得测试覆盖更加全面，可以覆盖到程序中所有可能的路径。由此可以看出只有对程序内部十分了解才能进行适度有效的覆盖测试。但是贯穿在程序内部的逻辑存在着不确定性和无穷性，尤其对于大规模复杂软件。因此我们不能穷举所有的逻辑路径，即使穷举也未必会带来好运（穷举不能查出程序逻辑规则错误，不能查出数据相关错误，不能查出程序遗漏的路径）。所以正确使用覆盖（白盒）测试，就要先从代码分析入手，根据不同的代码逻辑规则、语句执行情况，选用适合的覆盖方法。任何一个高效的测试用例，都是针对具体测试场景的。逻辑测试不是片面的测试正确的结果或是测试错误的结果，而是尽可能全面地覆盖每一个逻辑路径。这样才能使得测试的工作量与准确性达到一个合理的平衡。

24、针对提供软件产品和提供信息服务的这两类不同软件开发项目，讨论项目管理的特点。

软件产品开发的项目管理具有以下特点：

1、复杂性：软件项目管理具有高度的复杂性，涉及众多干系人、多变的需求、快速的技术更新和质量保障的挑战。项目涉及开发团队内部的不同角色，如程序员、测试人员和架构师，他们各自有不同的专业技能和工作重点。客户需求可能频繁变动，技术更新迅速，质量保障也需要严格的测试流程。

2、迭代性：迭代性是软件项目管理的另一个重要特点，特别是敏捷开发方法的广泛应用。迭代性强调反馈和调整，有助于分散风险和快速响应市场变化。敏捷开发方法如 Scrum 和 Kanban 帮助团队快速适应需求变化，提高项目的灵活性和适应性。

3、依赖性：软件项目管理依赖于多种技术和工具，需要在不同的环境中运行。项目经理需要协调多个团队和部门，确保每个部分都能顺利进行。技术复杂性要求团队成员具备不同的技能和经验，项目经理需要提供必要的培训和资源支持。

4、协作性：软件项目通常需要跨职能团队的协作，包括开发、测试、运维等多个部门。高效的团队协作工具如项目管理软件可以帮助团队成员高效沟通和协作。

成本控制：项目管理还需要关注人力、硬件设备和软件工具的成本。合理的资源分配和成本控制策略对于项目的成功至关重要。

应对这些特点的策略和方法包括：

（1）采用敏捷开发方法：如 Scrum 和 Kanban，快速响应需求变化，提高项目的灵活性和适

应性。
（2）使用项目管理软件：如 PingCode 和 Worktile，提供强大的团队协作功能，帮助团队成员高效沟通和协作。
（3）定期技术培训和学习：确保团队成员能够及时掌握新技术，保持项目的前沿性。
（4）严格的测试流程：包括单元测试、集成测试和系统测试，确保软件的质量和稳定性。
信息项目管理是指针对信息系统或信息技术相关项目的管理方法和技术。它与传统项目管理有许多相似之处，同时还具有一些独特的特点。
1. 复杂性：信息项目常常非常复杂，包括大量的技术、需求、资源和风险因素。信息系统往往与组织的各个部门和业务关联密切，涉及到大量的数据和流程。因此，信息项目管理需要对复杂性进行全面的分析和处理。
2. 技术驱动：信息项目通常是以信息技术为核心的，技术驱动是其独特的特点之一。在信息项目管理中，需要合理应用各种技术工具和方法，包括软件开发、数据库管理、网络技术等。同时，由于技术的快速更新和变化，信息项目管理需要具备灵活性和快速应对变化的能力。
3. 需求变化：信息项目中需求的变化是常态。由于信息系统常常涉及到大量的业务流程和数据，业务的变化会导致需求的变化。此外，用户需求也常常会在项目过程中被细化和调整。因此，信息项目管理需要具备需求变化管理的能力，包括对需求的及时响应和变更控制的技术手段。
4. 风险管理：信息项目常常伴随着一定的风险。信息系统的复杂性和技术驱动特点，使得项目面临着众多的风险因素，如技术风险、安全风险、成本风险等。信息项目管理需要建立完善的风险管理机制，对项目风险进行预测、评估和控制，同时制定相应的风险应对策略。
5. 沟通协作：信息项目管理需要广泛的沟通和协作。由于涉及到多个部门的合作，信息项目管理需要建立高效的沟通渠道，确保各方之间的需求和期望得到充分的明确和理解。此外，也需要建立项目团队间的良好协作机制，提高项目的整体效能。
为了应对以上特点，信息项目管理可以采用一些特定的方法和工具，如敏捷方法、原型开发、需求管理工具等。这些方法和工具能够提高信息项目的管理效率和质量，使项目达成预期目标。同时，信息项目管理也需要具备灵活性和创新性，及时应对项目中的变化和挑战。

18.论述软件项目特点和管理要点

软件项目的特点

复杂性：软件项目通常官由多个模块和组件组成，这些模块和组件之间存在复杂的相互作用关系。软件项目的开发和管理需要高度的技术和专业知识。

不确定性：软件项目的需求和规格说明通常是不完整、不准确和不一致的。此外，软件项目的开发过程中可能会出现各种问题的挑战，例如技术难题、人员变动、进度延误等。

可变性：软件项目的需求和规格说明可能会随着时间的推移而发生变化。因此软件项目管理需要具备灵活性和适应性，以便及时应对变化

风险性：软件项目的开发和管理涉及到多种风险，例如技术风险、进度风险、成本风险等。因此，软件项目管理需要具体风险管理能力，以便及时识别、评估和应对各种风险。

软件项目管理中，需要遵循一定的原则和方法，以确保软件项目的成功。具体而言，软

件项目管理需要考虑以下要点：

计划：制定详细的计划，并根据实际情况进行调整

进度控制：监控项目的进度，并及时采取措施解决问题

人员管理：合理分配人力资源，并保证团队协作

质量控制：确保软件产品符合质量标准，并及时修改缺陷

风险管理：识别、评估和应对各种风险

17.针对不同软件方法，论述项目管理要点

软件研究人员也在不断探索新的软件开发方法。至今已形成了八类软件开发方法.

一、Parnas 方法

此方法在 1972 年提出的。主要针对软件在可维护性和可靠性方面存在着严重问题，主要提出了信息隐蔽原则：在概要设计时列出将来可能发生变化的因素，并在模块划分时将这些因素放到个别模块的内部。第二条原则是在软件设计时应对可能发生的种种意外故障采取措施。软件是很脆弱的，很可能因为一个微小的错误而引发严重的事故，所以必须加强防范。。

二、SASA 方法

1978 年，**提出了结构化方法**，即 SASD 方法，也可称为面向功能的软件开发方法或面向数据流的软件开发方法，首先用结构化分析（SA）对软件进行需求分析，然后用结构化设计（SD）方法进行总体设计，最后是结构化编程（SP）。

三、**面向数据结构的**软件开发方法

1、Jackson 方法 ：从目标系统的输入、输出数据结构入手，导出程序框架结构，再补充其它细节，就可得到完整的程序结构图。此方法有时也称为面向数据结构的软件设计方法。

2、Warnier 方法

与 Jackson 方法类似。差别有三点：一是它们使用的图形工具不同，分别使用 Warnier 图和 Jackson 图；另一个差别是使用的伪码不同；最主要的差别是在构造程序框架时，Warnier 方法仅考虑输入数据结构，而 Jackson 方法不仅考虑输入数据结构，而且还考虑输出数据结构。

四、问题分析法

基本思想是：考虑到输入、输出数据结构，指导系统的分解，在系统分析指导下逐步综合。这一方法的具体步骤是：

从输入、输出数据结构导出基本处理框；分析这些处理框之间的先后关系； 按先后关系逐步综合处理框，直到画出整个系统的 PAD 图。PAM 方法的另一个优点是使用 PAD 图。这是一种二维树形结构图，是到目前为止最好的详细设计表示方法之一，远远优于 NS 图和 PDL 语言。

五、面向对象的软件开发方法

随着 OOP（面向对象编程）向 OOD（面向对象设计）和 OOA（面向对象分析）的发展，最终形成面向对象的软件开发方法 OMT(Object Modelling Technique)。这是一种自底向上和自顶向下相结合的方法，而且它以对象建模为基础，从而不仅考虑了输入、输出数据结构，实际上也包含了所有对象的数据结构。

六、可视化开发方法

用户界面在软件系统中所占的比例也越来越大，有的甚至高达 60～70%。产生这一问题的原因是图形界面元素的生成很不方便。可视化开发就是在可视开发工具提供的图形用户界面上，通过操作界面元素，诸如菜单、按钮、对话框、编辑框、单选框、复选框、列表框和

滚动条等，由可视开发工具自动生成应用软件。

七、ICASE

提高人类的劳动生产率，提高生产的自动化程度，一直是人类坚持不懈的追求目标。软件开发也不例外。

八、软件重用和组件连接

软件重用（Reuse）又称软件复用或软件再用。软件人工制品可以是源代码片断、子系统的设计结构、模块的详细设计、文档和某一方面的规范说明等。所以软件重用是利用已有的软件成份来构造新的软件。它可以大大减少软件开发所需的费用和时间，且有利于提高软件的可维护性和可靠性。

23、论述度量对软件工程的作用及存在的困难。

软件度量是指通过定量的测量方法来评估软件产品、过程和项目的属性或特征。它是软件工程中不可或缺的一部分，能够帮助开发者和管理者更好地了解软件的质量、进度和成本等方面的指标。本文将介绍软件度量的概念、分类和应用，并探讨其在软件工程中的重要性。

软件度量可以按不同的角度进行分类，常见的分类方式有以下几种：

1. 功能度量：评估软件的功能性能，如用户需求的满足程度、功能是否完备等。
2. 成本度量：衡量软件开发和维护的成本，包括人力、时间、物力等方面的评估。
3. 质量度量：评价软件的质量特性，如可靠性、可维护性、可测试性等。
4. 复杂度度量：评估软件的复杂性，包括代码行数、函数调用关系、模块依赖关系等。
5. 效率度量：评估软件的运行效率和资源利用率，如响应时间、内存占用等。

软件度量在软件工程中具有广泛的应用，以下是几个常见的应用领域：

1. 项目管理：通过对软件开发过程进行度量，可以评估项目的进度、风险和成本等，帮助项目经理制定合理的计划和管理策略。
2. 质量控制：通过度量软件的质量特性，可以及时发现和解决潜在的质量问题，提高软件的可靠性和稳定性。
3. 产品评估：通过对软件功能、性能和用户满意度等方面的度量，可以评估产品的优劣和市场竞争力。
4. 过程改进：通过度量软件开发过程中的各种指标，可以找出优化空间，改进开发方法和流程，提高软件开发效率。
5. 决策支持：通过对软件相关指标的度量，可以为管理层提供决策依据，帮助其制定战略和战术决策。

软件度量在软件工程中的重要性不可忽视。它不仅提供了对软件产品和过程的量化评估，还能够帮助开发团队和管理者发现问题、改进流程、提高质量和效率。通过精确的度量数据，可以更好地管理软件项目，减少风险，并为决策提供科学依据。

软件度量中遇到的困难主要包括以下几个方面：

（1）需求不清晰或变动频繁：在很多项目中，需求的不明确或频繁变动是影响测试质量和效率的一个常见痛点。如果在项目初期，需求没有得到充分确认或经常变动，测试人员难以根据准确需求来制定测试计划，容易导致测试用例设计不充分、测试执行无效等问题。

（2）测试环境不稳定或配置不一致：测试环境与生产环境的不一致，或者测试环境的不稳定，是许多测试项目中的隐形痛点。测试人员无法在与真实生产环境相似的环境中执行测试，

导致测试结果不准确、测试周期延长。
（3）测试覆盖不足：时间紧迫和资源不足是测试团队面临的常见难题。在这种情况下，测试的覆盖面会被压缩，导致很多边界条件、异常情况得不到验证，从而埋下了隐患。
（4）开发与测试之间的沟通障碍：开发和测试团队之间的沟通不畅，常常导致工作效率低下、重复劳动，甚至项目延误。测试人员和开发人员在需求理解、Bug 反馈、版本更新等方面的沟通不及时，往往会导致测试遗漏、开发任务重复等问题。
（5）战略目标与测量程序的对齐问题：在软件工程组织中，战略目标的制定与测量程序的设计需要高度一致。研究中发现，学生在理解如何将战略目标与测量程序正确对齐方面存在困难。
（6）测量程序的效率问题：许多学生在设计指标时遇到了时间管理上的困难，导致需要的时间是其他团队的两倍。这主要是因为他们没有将测量结果分类和组织，影响了指标设计和获取结果的方法的理解。
（7）测量信息的效用问题：研究还发现学生在理解方法信息的用途方面存在明显不足，尤其是缺乏对战略管理的了解。这影响了他们对方法目的的理解，特别是将战略目标与指标对齐的能力。

22、对于人的健康来讲“预防为主”比较好理解，软件工程也主张适当提高预防成本来降低整体的质量成本，对于软件来讲如何体现预防为主的思想?

一、引言
在当今的信息化时代，软件已经成为企业和个人日常生活中不可或缺的一部分。软件项目的质量不仅关系到用户的体验和满意度，还直接影响企业的生存和发展。因此，预防软件项目质量问题至关重要。本篇文档将详细介绍软件项目质量预防措施，帮助企业和开发团队提高软件质量，降低项目风险。
二、软件项目质量概述
软件项目质量是指软件产品满足用户需求和期望的程度。软件项目的质量取决于多个因素，包括需求分析、设计、编码、测试、部署等。在软件开发过程中，任何一个环节的失误都可能导致软件项目质量的下降。
三、软件项目质量预防措施
1.需求分析阶段
需求分析是软件项目的基础,准确的需求分析是保证软件项目质量的关键。在需求分析阶段，应采取以下预防措施：
（1）充分了解用户需求和期望，包括功能需求、性能需求、安全需求等。
（2）与用户进行充分沟通，确保对用户需求的理解无误。
（3）对需求进行详细的分析和评估，确保需求的合理性和可行性。
（4）制定需求变更应对策略，以应对可能的需求变更。
2.设计阶段
设计阶段是软件开发过程中至关重要的环节，良好的设计可以避免后期开发过程中的错误和返工。在设计阶段，应采取以下预防措施：
（1）根据需求分析结果，制定相应的设计文档和架构。
（2）遵循设计原则和最佳实践，确保设计的合理性和可维护性。
（3）进行设计评审，邀请领域专家和经验丰富的开发人员对设计进行评估和改进。

（4）在设计阶段引入技术预研，以便更好地选择适合的技术和工具。

3.编码阶段

编码阶段是实现软件设计的关键环节，也是最容易出现错误的环节之一。在编码阶段，应采取以下预防措施：

（1）遵循编码规范和标准，确保代码的可读性和可维护性。

（2）引入代码审查机制，对代码进行审核和把关。

（3）采用适当的开发方法和工具，提高代码质量和生产效率。

（4）重视代码测试和调试，确保代码的正确性和稳定性。

4.测试阶段

测试是保证软件项目质量的必要环节。在测试阶段，应采取以下预防措施：

（1）制定详细的测试计划和用例，确保测试的全面性和有效性。

（2）采用自动化测试工具和框架，提高测试效率和准确性。

（3）进行性能测试、安全测试等多种类型的测试，确保软件项目的各个方面的质量。

（4）及时处理和报告测试中发现的问题，进行缺陷跟踪和管理。

5. 部署阶段

部署是将软件项目发布到生产环境的关键环节。在部署阶段，应采取以下预防措施：

（1）制定详细的部署计划和流程，确保部署的顺利进行。

（2）遵循配置管理和版本控制原则，避免出现配置混乱和版本冲突等问题。

（3）进行生产环境的监控和维护，确保软件的稳定性和安全性。

（4）及时收集用户反馈和意见，以便对软件项目进行持续改进和优化。

6. 项目管理阶段

项目管理是保证软件项目顺利实施的关键环节。在项目管理阶段，应采取以下预防措施：

（1）制定合理的项目计划和时间表，确保项目按期完成和质量达标。

（2）采用敏捷开发方法或传统瀑布模型等方法进行项目管理，确保项目管理的有效性和灵活性。

（3）进行项目风险管理，及时识别和应对潜在的项目风险。

（4）加强团队沟通和协作，提高团队凝聚力和工作效率。

6.什么是软件可维护性，讨论软件工程与可维护性关系

答：软件可维护性即维护人员对该软件进行维护的难易程度,具体包括理解、改正、改动和改进该软件的难易程度。

软件工程是一门研究用工程化方法构建和维护有效的、实用的和高质量的软件的学科。

软件工程的目标是：在给定成本、进度的前提下，开发出具有适用性、有效性、可修改性、可靠性、可理解性、可维护性、可重用性、可移植性、可追踪性、可互操作性和满足用户需求的软件产品。软件交付使用后，需要能够对它进行修改，以改正潜伏的错误，改进性能和其它属性，使软件产品适应环境的变化等。软件维护费用在软件开发费用中占有很大的比重。因此，**可维护性是软件工程中一项十分重要的目标**。

14.针对敏捷方法的十二条核心实践，结合软件工程问题论述其有效性

PPT 参考答案：

1. On-Site Customer （现场客户） : 客户是 Team 成员，在开发现场和开发人员一起工作。

2. 计划项目 （Planning Game）

3. 频繁地小规模发布软件（Small Releases）

4. 简单设计 （Simple Design）

5. 测试驱动开发 （Test Driven Development）

6. 持续集成 （Continuous Integration）

7. 集体拥有代码（Collective Code Ownership）

8. 编程规范（Coding Standards）

9. 重构（Refactoring）

10. System Metaphor（系统隐喻）

11. Pair Programming（结对编程）

12. 平稳的工作效率（Sustainable Pace）

敏捷开发中有 12 条原则，它们是敏捷实践区别于重型过程的特征所在。

1.我们最优先要做的是通过尽早的、持续的交付有价值的软件来使客户满意。

2.即使到了开发的后期，也欢迎改变需求。敏捷过程利用变化来为客户创造竞争优势。

3.经常性的交付可以工作的软件，交付的间隔可以从几周到几个月，交付的时间间隔越短越好。

4.在整个项目开发期间，业务人员和开发人员必须天天都在一起工作。

5.围绕被激励起来的人来构建项目。给他们提供所需要的环境和支持，并且信任他们能够完成工作。

6.在团队内部，最具有效果并且富有效率的传递信息的方法，就是面对面的交谈。

7.工作的软件是首要进度度量标准。

8.敏捷过程提可持续的开发速度。责任人、开发者和用户应该能够保持一个长期的、恒定的开发速度。

9.不断地关注优秀的技能和好的设计会增强敏捷能力。

10.简单——使未完成的工作最大化的艺术——是根本的。

11.最好的构架、需求和设计出自与自组织的团队。

12.每隔一定时间，团队会在如何才能更有效地工作方面进行反省，然后相应地对自己的行为进行调整。

19.论述软件估算的困难和解决方法

软件成本估算是对将要开发的或正在开发的软件项目所需要的工作量和工作进度作出预测，从而产生出一组在可接受误差范围内的近似规划。

成本估算的困难：

1） 需求信息的复杂性：与其他有些传统项目不同，信息系统要满足的人的主观需要。由于人的复杂性，给信息系统带来了无数的难以确定的因素。而且，随着项目的进展，许多具体情况的明确，项目的成本估算也会相应的有所变化。

2）开发技术和工具的不断变化：开发工具软件的不断升级，技术方案不断更新，这些技术的进步让信息系统项目可以提供功能越来越强,但是给信息系统项目的成本估算带来困难。

3）缺乏类似项目估算数据可供参考：有效的项目成本估算是建立在大量的同类项目的成本估算的基础上的。没有大量的同类项目的经验，信息系统项目的成本估算也就非常困难了。

4）缺乏专业和富有经验的人才。

5）信息系统研发人员技术能力的差异：不同人员的不同态度、经验和能力都会造成不同人员的截然不同的效率，这也给信息系统的成本估算带来了困难。

6.）管理层的压力与误解。

成本估算的方法：

1) 工作分解结构方法（WBS）

2) 专家判断

3) 类比估算

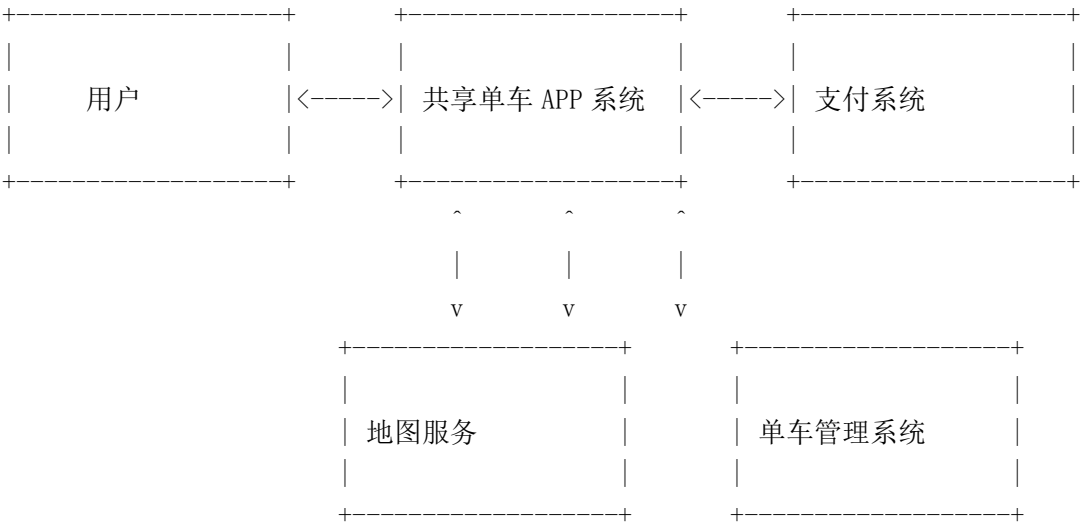
4) 其他估算方法：德尔菲法(Delphi technique)、COCOMO 模型、特征点(feature point)、对象点(object point)、3-D 功能点(3-D function points，数据、功能、控制)、Bang 度量(DeMarco‘s bang metric)、模糊逻辑(fuzzy logic)、标准构件法(standard component)等

5) 基于问题的估算

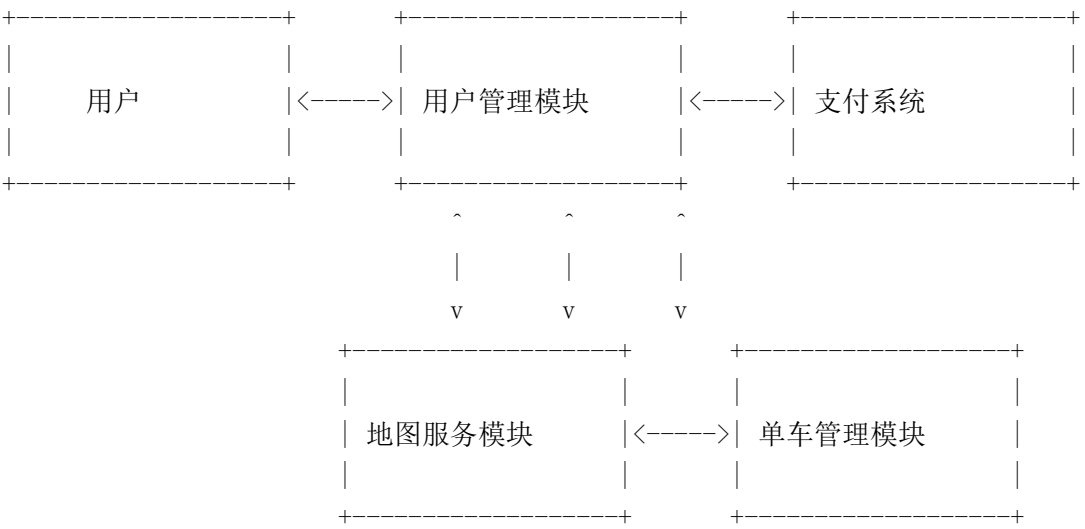
21.共享单车 APP 作为日常出行租用自行车的软件，旨在给人们提供一个优雅、方便、时尚的方式，解决了最后一公里。共享单车 APP 的功能如下：

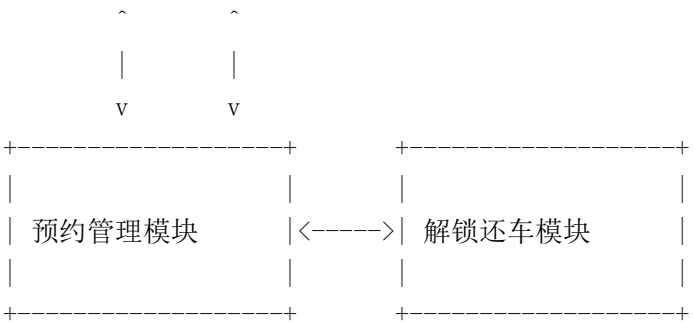
1.实时车辆地图：车辆实时定位，导航引导到达。
2.预约功能：提前预约周边单车，掌控行程安排。
3.一键扫码解锁：扫码车身二维码，即可解锁用车。
4.随时随地轻松还车：无固定车桩，在任意公共停车区域即可停车。
5.在线支付：提供多种电子支付方式。
要求用结构化分析方法（SA）建模：
1.画出系统的顶层（0 层）和第一层数据流图。
2.列出主要的数据字典（DD）项，包括源点和汇点，主要数据流、数据文件。

顶层（0 层）数据流图
顶层数据流图展示了系统与外部实体之间的交互。外部实体包括用户、支付系统、地图服务等。



第一层数据流图
第一层数据流图展示了系统内部的主要处理过程和数据流。





主要的数据字典（DD）项

源点和汇点

源点：

用户：提供输入数据，如预约请求、扫码请求、支付请求等。

地图服务：提供实时地图数据和导航信息。

单车管理系统：提供单车状态信息（如位置、可用性等）。

汇点：

用户：接收输出数据，如地图显示、预约确认、支付结果等。

支付系统：接收支付请求并返回支付结果。

单车管理系统：接收单车状态更新（如解锁、还车等）。

主要数据流

用户输入：

预约请求：用户提交的预约单车的请求。

扫码请求：用户扫描单车二维码的请求。

支付请求：用户提交的支付请求。

系统输出：

地图显示：显示实时单车位置和导航信息。

预约确认：确认用户的预约请求。

支付结果：返回支付成功或失败的结果。

内部数据流：

单车状态：单车的实时状态信息（如位置、可用性等）。

用户信息：用户的账户信息、支付信息等。

数据文件
用户信息文件：

存储用户的账户信息、支付信息、历史记录等。

单车信息文件：

存储单车的实时状态信息（如位置、可用性、故障状态等）。

预约记录文件：

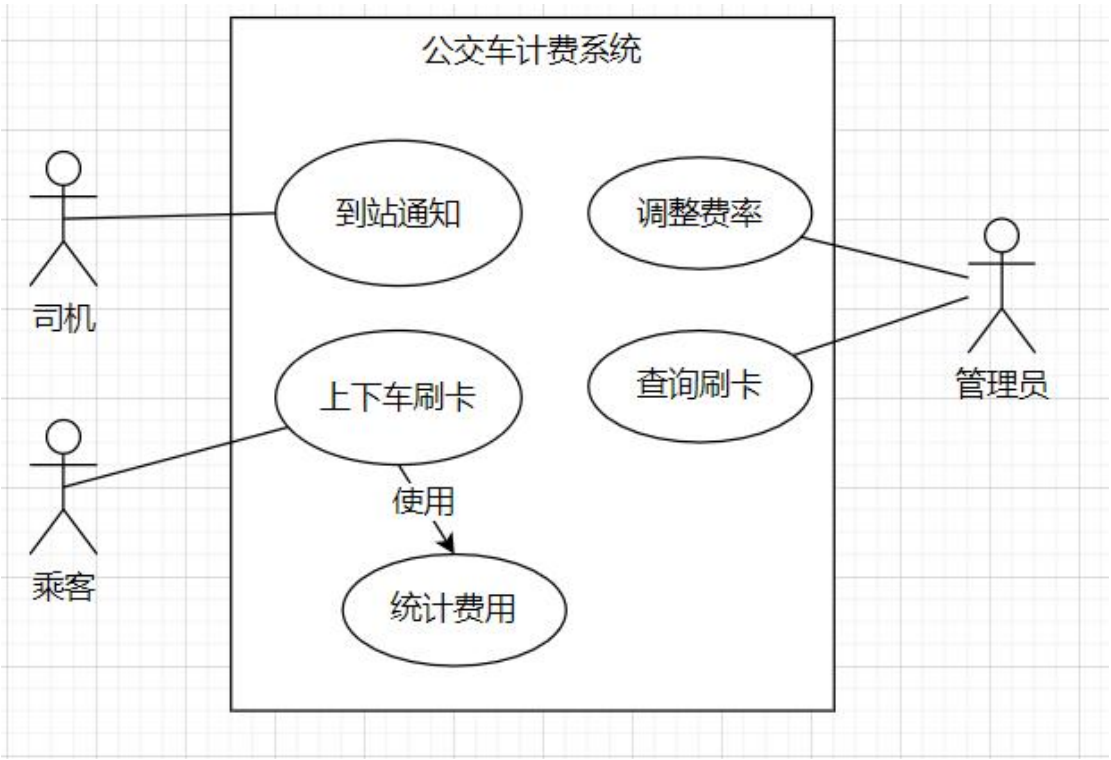
存储用户的预约记录，包括预约时间、单车编号、用户 ID 等。

支付记录文件：

存储用户的支付记录，包括支付时间、金额、支付方式等。

22、一个公交车计费系统，司机到站点击到站按钮，通知系统到站，乘客上车刷卡，下车刷卡，乘务员查询乘客是否刷卡，统计费用，管理员调整费率，计费方式

（1）画用例图
（2）画类图
（3）画时序图
答：
（1）用例图

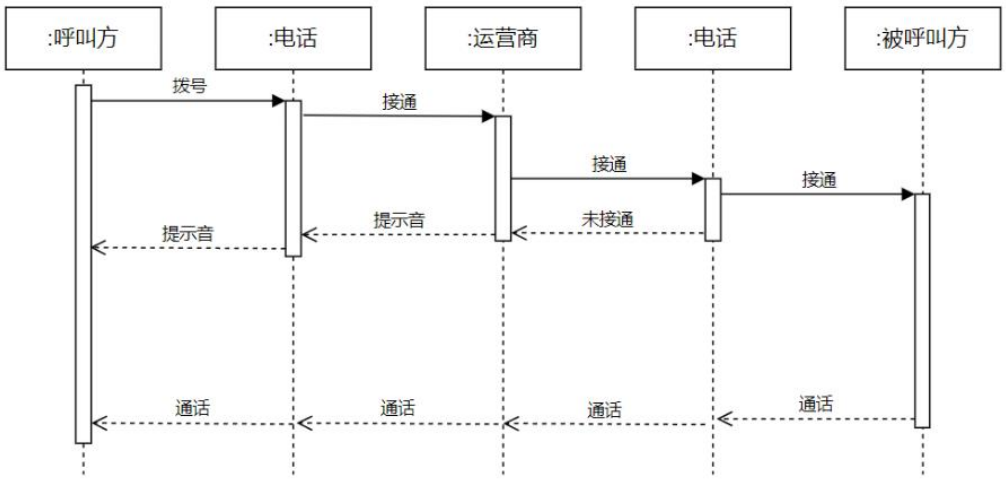

公交车计费系统
到站通知
调整费率
上下车刷卡
查询刷卡
使用
统计费用
司机
乘客
管理员

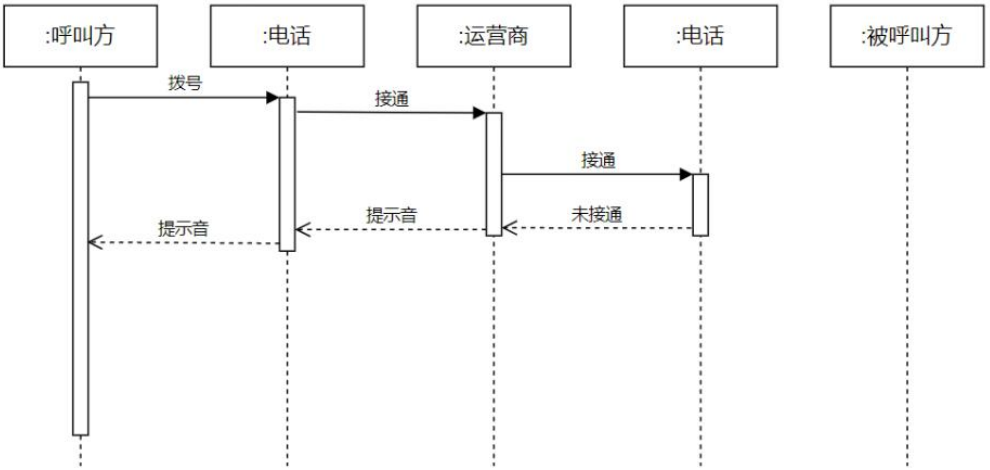
（2）类图：


司机
+ 姓名
+ 开车
乘客
+ 上车站点
+ 下车站点
+ 费率
+ 统计费用
管理员
+ 调整费率
+ 调整计费方式
+ 查询刷卡情况

（3）时序图（顺序图）

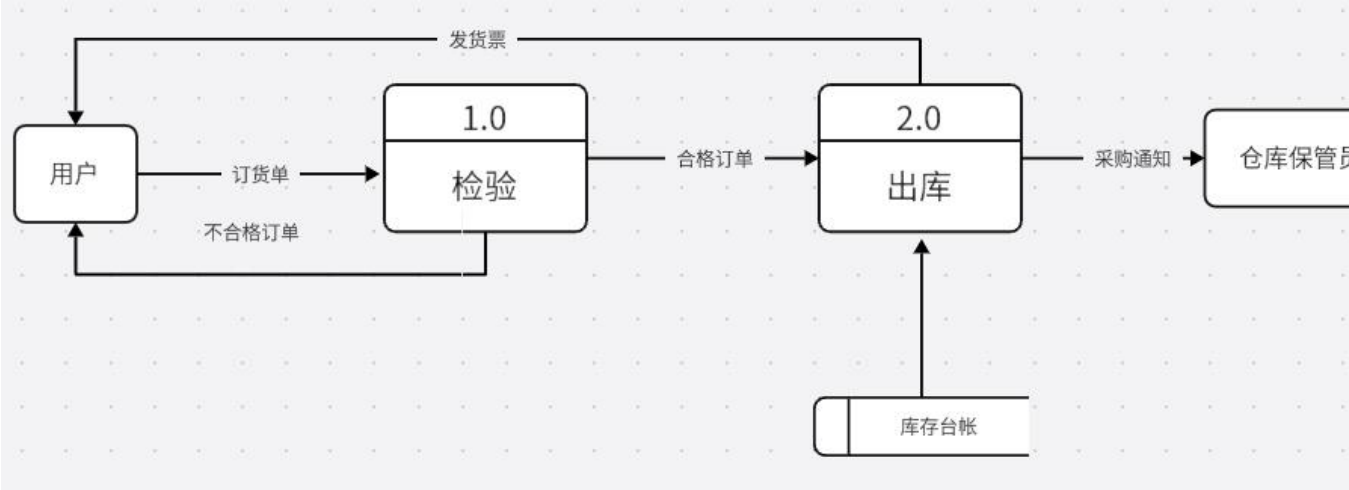
23、A 和 B 打电话-时序图（顺序图） ①打通电话 ②未打通电话





24、（练习）根据以下内容画出数据流图

（1）请根据以下销售过程画出数据流程图:用户将定货单交给某企业的业务经理，经检验后，对不合格的订单要由用户重填:合格的订单交仓库保管员做出库处理，即:查阅库存台帐，如果有货则向用户开票发货，如缺货，则通知采购员采购。



（2）请根据以下定货过程画出数据流程图采购员从仓库收到缺货通知单后立即进行定货处理，即，查阅订货合同单，若已定货，则向供货单位发出催货单，否则，填写定货单送供货单位。供货单位发运货物后，立即向采购员发出取货通知。

